

Genomics Data Visualization

with Python · Day 2 · Genome-scale charts
& comparative views

Dr. Loukas Theodosiou // Day 2 // June 2026

2026-06-23

Welcome to Day 2

GENOME-SCALE CHARTS · DAY 02

What We'll Build Today

OUTLINE • DAY 02

From single genes to whole genomes

Yesterday we plotted single experiments and single contrasts. Today we *zoom out*: the unit of analysis is no longer one gene or one sample, it is **the genome itself**. Each chart in this deck answers a question that only makes sense at that scale.

- **Stacked bars** — composition of mixtures (admixture, microbiome)
- **Time series** — trajectories with uncertainty
- **MSA** — letters that line up across sequences
- **Manhattan** — significance across the whole genome
- **Genome stats** — coverage, peaks, gene models
- **pyCirclize** — circular genome plots
- **Synten** — comparing genomes side-by-side

TODAY'S PROMISE

Every chart starts from a tidy `pandas` DataFrame and grows **one annotated line at a time**. Read the comments — they're the lesson.

The Setup Cell

LOGISTICS · DAY 02

One install, then everything works

`genomics_course` ships every loader, the Economist theme, and the matplotlib `rcParams`. Run this once per Colab/Jupyter session, then every later cell just imports.

```
1 pip install -q "git+https://github.com/loukesio/dataviz-genomicsdata.git@Python_2026"
```

```
1 # the theme registers IBM Plex Mono, defines the Economist palette
2 # (GREEN, BLUE, AMBER, RED, PURPLE + neutrals), and applies the
3 # cream-canvas matplotlib rcParams. One import, deck-wide consistency.
4 from genomics_course.theme import *
5
6 from genomics_course.data import (
7     load_gwas, load_admixture, load_gc,
8     load_peaks, load_peak_genes, load_timecourse,
9 ) # the loaders we'll use today
10
11 import numpy as np, pandas as pd
12 import matplotlib.pyplot as plt, seaborn as sns
13 import matplotlib.patches as mpatches # for synteny ribbons later
```

Stacked Bar Plots

COMPOSITION · PROPORTIONS

What Is a Stacked Bar?

STACKED BAR · THEORY

Every column sums to the whole; bands stack to show its parts

A **stacked bar** answers one question: *what is each sample made of?* Each column is a single observation (an individual, a tumour, a microbiome sample). The bar is sliced into coloured **bands**, one per category, and the heights of those bands sum to the total. When every bar has the same total — say, a proportion that must add to 100% — the chart becomes a pure **composition** view.

Use it when:

- **Admixture / ancestry:** each individual = one bar made of ancestry fractions K1...K5.
- **Microbiome:** each sample = one bar of phylum or genus relative abundance.
- **Cell-type deconvolution:** each tumour = one bar of immune-cell fractions.

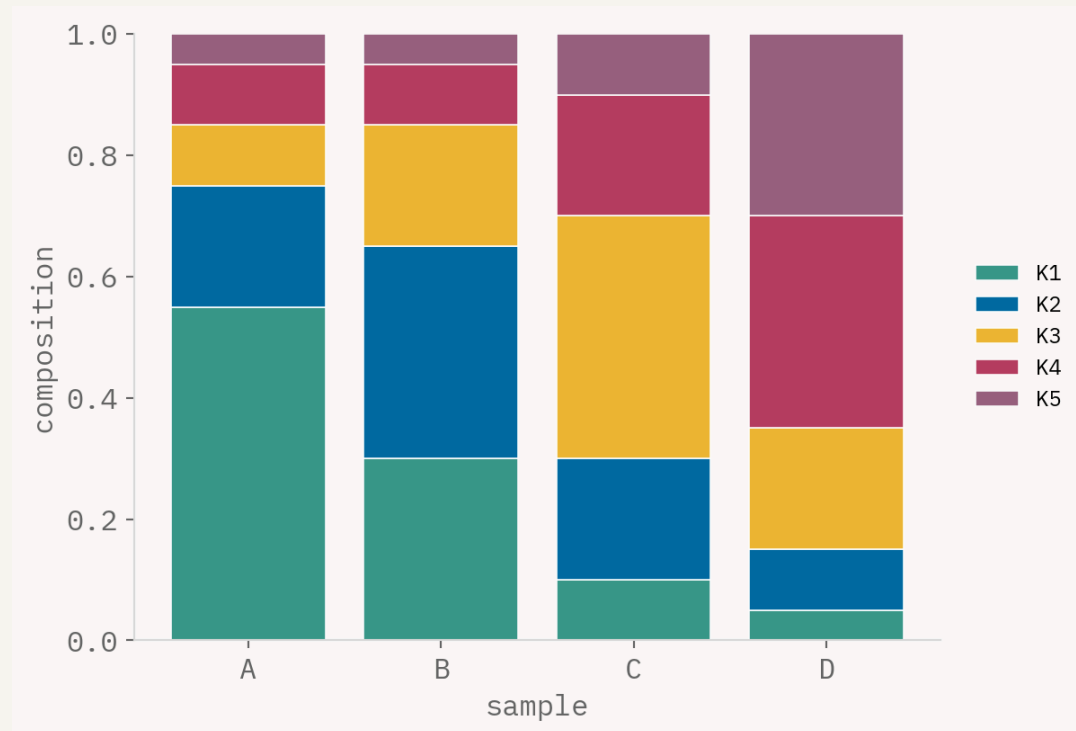
Avoid it when only the totals matter (a plain bar chart is clearer) or when you need to *compare a single inner band across many samples* (segment baselines wobble — a line, dot plot, or small multiples wins).

Anatomy, Before Any Data

STACKED BAR • THEORY

Four bars, five bands — how the geometry works

Before touching real ancestry data, build the **idea**. Four cartoon bars, each made of five rectangles in the course palette. The trick is the `bottom=` argument: each band starts where the previous one ended.



Stacked Bar • The Data

STACKED BAR • DATA PEEK

One row per individual — population label + five ancestry fractions

`load_admixture()` returns 120 individuals across five reference populations (EUR / EAS / AFR / AMR / SAS). Each row carries five ancestry components **K1...**
K5 whose values sum to 1.0 — they are mixture proportions, the output of a tool like ADMIXTURE or STRUCTURE.

```
1 adm = load_admixture()           # individual, population, K1..K5 (K1..K5 sum to 1)
2 adm.head(5)
```

	individual	population	K1	K2	K3	
0	EUR_000	EUR	0.978699	0.002754	0.000058	0.000495
1	EUR_001	EUR	0.978085	0.011594	0.000160	0.000160
2	EUR_002	EUR	0.977956	0.009453	0.000056	0.000056
3	EUR_003	EUR	0.971030	0.001839	0.025570	0.000056

Stacked Bar · Step 1 — One Individual

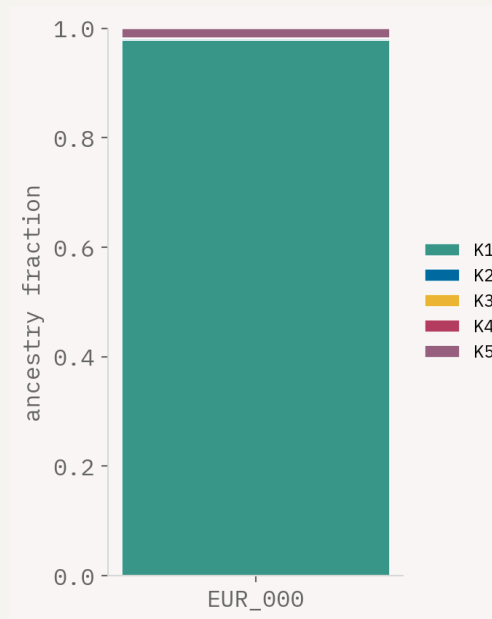
STACKED BAR · BUILD

1 · A single bar, K1...K5 stacked on top of each other

Build the **idea** on one person. Take row 0 (an EUR individual), stack their five K fractions vertically with

`ax.bar(..., bottom=running_total)`. The bar height ends at exactly 1.0 because the proportions sum.

```
1 adm = load_admixture()           # the full ancestry table
2 row = adm.iloc[0]                # one individual (the first
3
4 # the five mixture proportions for this person
5 k_cols = ["K1", "K2", "K3", "K4", "K5"] # column names in order
6 fracs = [row[k] for k in k_cols]      # numeric values, sum to 1.
7
8 fig, ax = plt.subplots(figsize=(3.8, 4.6)) # CREATE the canvas
9 bottom = 0                         # start the stack at y = 0
10
11 for i, k in enumerate(k_cols):      # one band per K component
12     ax.bar(row["individual"], fracs[i], # CALL: draw this band
13           bottom=bottom,              # stack it atop the previc
14           color=COURSE_PAL[i], edgecolor="white", linewidth=0.6,
15           label=k)
16     bottom += fracs[i]              # next band starts here
17
18 ax.set_ylim(0, 1); ax.set_ylabel("ancestry fraction")
19 ax.spines[["top", "right"]].set_visible(False)
20 ax.legend(loc="center left", bbox_to_anchor=(1.02, 0.5), frameon=False, font
```



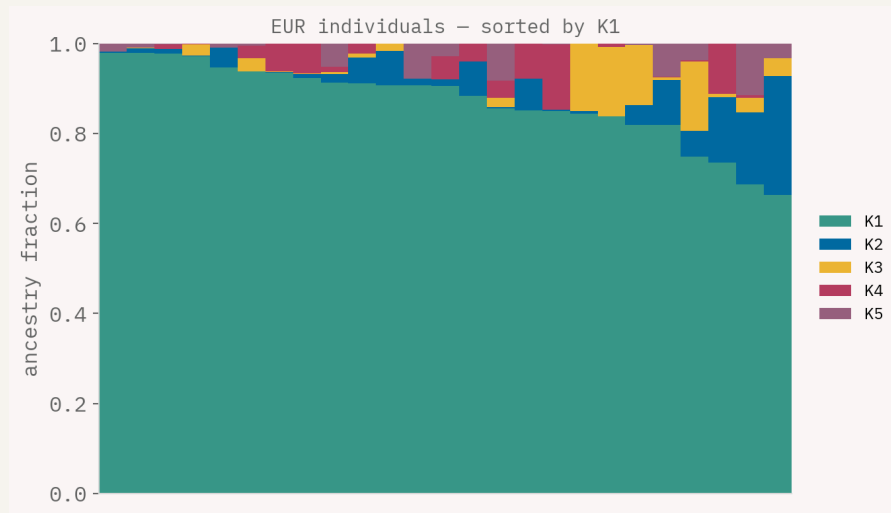
Stacked Bar · Step 2 — One Population, Sorted

STACKED BAR · BUILD

2 · All EUR individuals, sorted by their K1 contribution

Now scale to every EUR individual. Sort them by **K1** so the dominant ancestry forms a smooth gradient — without sorting, the bars look like noise. The `for` loop is unavoidable here: matplotlib's `ax.bar` needs the running `bottom` updated per band.

```
1 adm = load_admixture() # full table
2 k_cols = ["K1", "K2", "K3", "K4", "K5"]
3
4 eur = adm[adm["population"] == "EUR"].copy() # one population
5 eur = eur.sort_values("K1", ascending=False) # gradient by K1
6
7 x = np.arange(len(eur)) # 0, 1, 2, ...
8 bottom = np.zeros(len(eur)) # running total
9
10 fig, ax = plt.subplots(figsize=(7.2, 4.2)) # CREATE the plot
11
12 for i, k in enumerate(k_cols): # one band at a time
13     heights = eur[k].values # bracket ancestry
14     ax.bar(x, heights, bottom=bottom, # CALL: draw bar
15           color=COURSE_PAL[i], width=1.0, # width=1.0
16           edgecolor="none", label=k)
17     bottom += heights # advance bottom
18
19 ax.set_xlim(-0.5, len(eur) - 0.5); ax.set_ylim(0, 1)
20 ax.set_xticks([]); ax.set_ylabel("ancestry fraction")
21 ax.set_title("EUR individuals - sorted by K1", color=MUTED, fontsize=11)
22 ax.spines["top"] = ax.spines["right"] = ax.spines["left"] = False
```



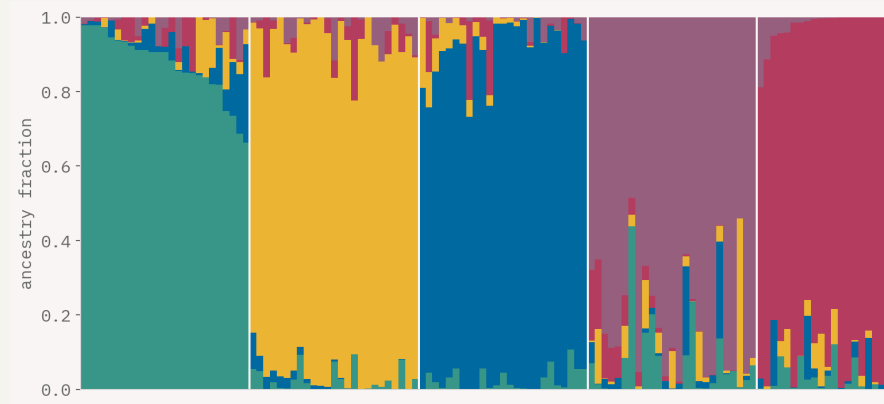
Stacked Bar · Step 3 — Every Population

STACKED BAR · BUILD

3 · All five populations, separators between groups

The canonical admixture plot. Sort *within* each population by its dominant K, concatenate, then draw thin vertical lines between groups so the eye can find the boundaries.

```
1 adm = load_admixture() # full i
2 k_cols = ["K1", "K2", "K3", "K4", "K5"]
3 pops = ["EUR", "EAS", "AFR", "AMR", "SAS"] # fixed
4 dom = {"EUR": "K1", "EAS": "K2", "AFR": "K3", "AMR": "K4", "SAS": "K5"}
5
6 # sort each population internally by its dominant K, then concatenate
7 ordered = pd.concat([
8     adm[adm["population"] == p] # one i
9     .sort_values(dom[p], ascending=False) # grad:
10     for p in pops
11 ], ignore_index=True)
12
13 x = np.arange(len(ordered)) # slot
14 bottom = np.zeros(len(ordered))
15
16 fig, ax = plt.subplots(figsize=(9.0, 4.2)) # CREA!
17
18 for i, k in enumerate(k_cols): # paint
19     heights = ordered[k].values
20     ax.bar(x, heights, bottom=bottom, # CALL
21           color=COURSE_PAL[i], width=1.0, edgecolor="none", label=k)
22     bottom += heights
```



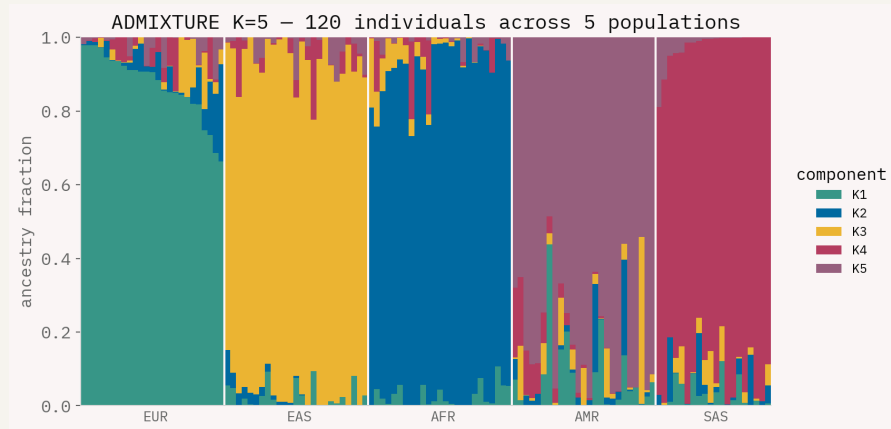
Stacked Bar · Step 4 — Labels & Legend

STACKED BAR · BUILD

4 · Population names along the x, K legend on the right

The composition is right; now make it readable. Centre each population's name *under its block* of bars using the cumulative counts, and pin the K legend outside the axes so it never overlaps the data.

```
1 adm = load_admixture()
2 k_cols = ["K1", "K2", "K3", "K4", "K5"]
3 pops = ["EUR", "EAS", "AFR", "AMR", "SAS"]
4 dom = {"EUR": "K1", "EAS": "K2", "AFR": "K3", "AMR": "K4", "SAS": "K5"}
5
6 ordered = pd.concat([
7     adm[adm["population"] == p].sort_values(dom[p], ascending=False)
8     for p in pops
9 ], ignore_index=True)
10
11 x = np.arange(len(ordered))
12 bottom = np.zeros(len(ordered))
13
14 fig, ax = plt.subplots(figsize=(9.0, 4.4))
15
16 for i, k in enumerate(k_cols):
17     heights = ordered[k].values
18     ax.bar(x, heights, bottom=bottom,
19           color=COURSE_PAL[i], width=1.0, edgecolor="none", label=k)
20     bottom += heights
21
22 counts = ordered["population"].value_counts().reindex(pops)
```



Stacked Bar • Pandas Variant

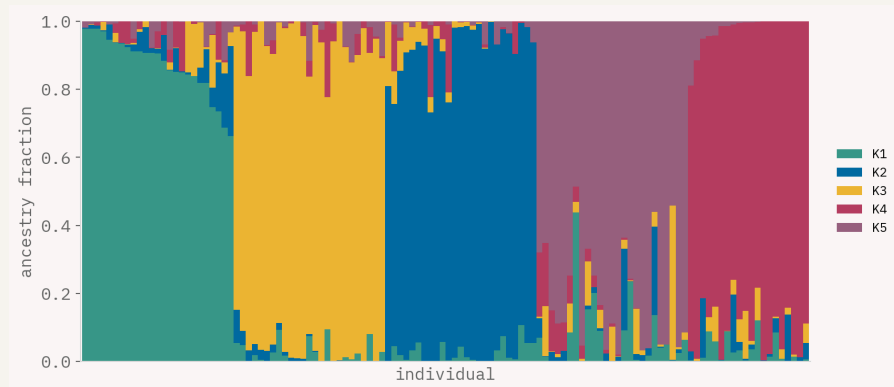
STACKED BAR • ERGONOMIC

Three lines with `pandas.DataFrame.plot(stacked=True)`

Pandas wraps matplotlib for the common case. Set the population-ordered index on the K columns and ask for a stacked bar — pandas computes the running `bottom=` for you. You give up control (no per-population sort across groups, harder custom separators), but you get to a publishable draft in three lines.

```
1 adm = load_admixture()
2 k_cols = ["K1", "K2", "K3", "K4", "K5"]
3 pops = ["EUR", "EAS", "AFR", "AMR", "SAS"]
4
5 # group-ordered frame: each pop sorted by its dominant K, indexed by individ
6 ordered = pd.concat([adm[adm["population"] == p].sort_values(k_cols[i], asce
7                     for i, p in enumerate(pops)], ignore_index=True)
8 data = ordered.set_index("individual")[k_cols]          # rows=ind, cc
9
10 # CALL: pandas builds the whole stacked bar from the frame
11 ax = data.plot(kind="bar", stacked=True, width=1.0,
12               color=COURSE_PAL, figsize=(9.0, 4.0), legend=True)
13 ax.set_xticks([]); ax.set_ylim(0, 1); ax.set_ylabel("ancestry fraction")
14 ax.spines[["top", "right"]].set_visible(False)
15 ax.legend(loc="center left", bbox_to_anchor=(1.02, 0.5), frameon=False, font
```

Trade-off: pandas wins on conciseness; matplotlib wins on control — custom separators, per-population sort



orders, labels at block midpoints. Use pandas to **draft**,
matplotlib to **publish**.

Stacked Bar • Takeaway

STACKED BAR • RECAP

Composition first, totals only if they vary

THE RECIPE

1. **Each column is one observation**; bands stack to its total. 2. **Sort within groups** by the dominant component — un-sorted admixture looks like noise. 3. **One colour per band, fixed across bars** — the eye tracks a band by its colour. 4. **Separators between groups** (thin cream lines) replace heavy x-ticks.

When the totals are forced equal (admixture proportions sum to 1), the stacked bar is **pure composition**. When totals vary (read counts, dollars), it shows **composition and magnitude** in one chart. Pick deliberately.

Time Series Plots

TRAJECTORIES WITH UNCERTAINTY

What Is a Time Series Plot?

TIME SERIES · THEORY

x is time, y is the measurement, lines show trends, bars or bands show uncertainty

A **time series plot** answers two questions at once: *what is the trajectory*, and *how confident are we at each point*. The x-axis carries time (minutes, hours, days, weeks), the y-axis carries the measurement (expression, abundance, signal). One **line per entity** traces its trajectory — gene, sample, cell line, patient.

The interesting question is always **how to show uncertainty** around each point:

- **Error bars** (`ax.errorbar`): vertical whiskers $\pm$ SEM or 95% CI at every measured time. Best when you have **few points** (≤ 6 timepoints) — the eye can read each whisker.
- **Ribbons** (`ax.fill_between`): a translucent band between mean $\pm$ SEM. Best when you have **many points** (a dense kinetic curve) — error bars would clutter; a ribbon shows the envelope at a glance.
- **Small multiples**: one panel per entity, faint grey context behind each focal trajectory. Best when you have **many entities** and lines overlap.

The rule is: pick the encoding that lets the audience *read direction + confidence* without effort.

Time Series • The Data

TIME SERIES • DATA PEEK

One row per (gene, timepoint) — tpm with its standard error

`load_timecourse()` returns a tidy long-form table: 6 genes (BRCA1, TP53, EGFR, MYC, KRAS, PTEN) measured at 5 timepoints (0, 2, 6, 12, 24 hours), with the mean **tpm** and its standard error **sem**. Long-form is the format every plotting library expects.

```
1 tc = load_timecourse()      # gene, time, tpm, sem (long-form, 6 genes x 5 tim
2 tc.head(6)
```

	gene	time	tpm	sem
0	BRCA1	0	13.0	3.062475
1	BRCA1	2	15.0	1.901278
2	BRCA1	6	17.0	2.320425
3	BRCA1	12	17.0	2.690532
4	BRCA1	24	23.0	3.510216
5	TP53	0	23.0	3.334770

Time Series • The `.sem` Footgun

TIME SERIES • GOTCHA

`DataFrame.sem` is a *method*. Always use bracket access for the column

This dataset has a column literally called `sem` — the standard error of the mean. **But** pandas DataFrames already have a `.sem()` method (it computes the standard error). Dot access loses to the method every time and you get a confusing error. The fix is one character: brackets.

The bug — `tc.sem` returns the **method**, not the column:

```
1 tc = load_timecourse()
2
3 tc.sem                # bound method, not data
4 # <bound method ... DataFrame.sem of ...>
5
6 tc.sem.values         # AttributeError!
7 # AttributeError: 'function' object
8 # has no attribute 'values'
```

When you pass `yerr=tc.sem` to matplotlib you get a cryptic `TypeError` because matplotlib tried to use the method as an array.

The fix — always use bracket access:

```
1 tc = load_timecourse()
2
3 tc["sem"]             # the actual column
4 # 0    3.062
5 # 1    1.901
6 # ...
7
8 tc["sem"].values      # plain numpy array
9 # array([3.06, 1.90, 2.32, ...])
```

Rule of thumb: prefer `df["col"]` over `df.col` always. Brackets work for every legal name; dots silently break on `mean`, `sum`, `sem`, `count`, `index`, `T`, ...

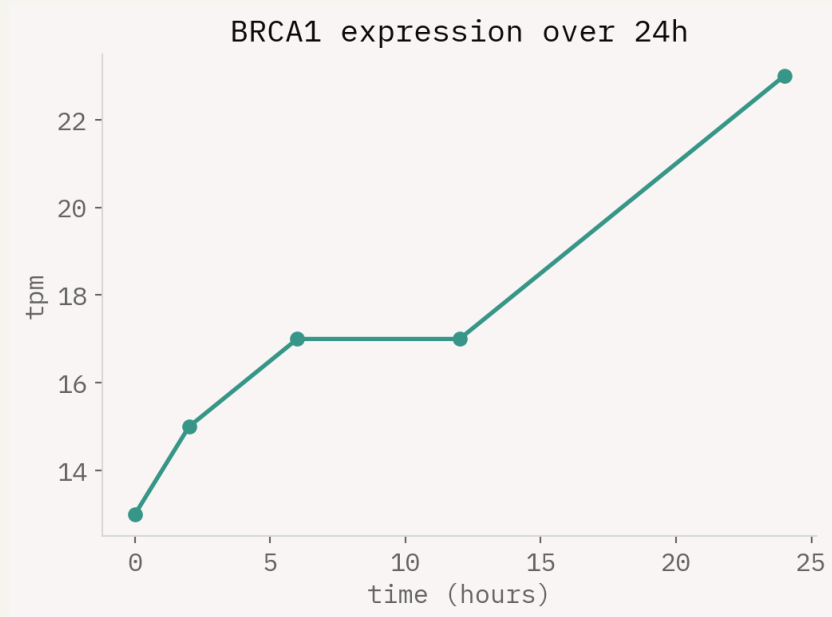
Time Series • Step 1 — One Gene

TIME SERIES • BUILD

1 • BRCA1 only, time vs tpm

Build the **idea** on one gene. Filter the long table to BRCA1, draw time on x and tpm on y. A single `ax.plot` with markers tells the trajectory.

```
1 tc = load_timecourse()           # the full long-form tab
2 brca1 = tc[tc["gene"] == "BRCA1"] # filter to one gene
3
4 # sanity check: 5 rows, one per timepoint
5 brca1 = brca1.sort_values("time") # ensure x is monotonic
6
7 fig, ax = plt.subplots(figsize=(5.6, 4.2)) # CREATE the canvas
8 ax.plot(brca1["time"], brca1["tpm"],      # CALL: x = time, y = tpm
9         marker="o", color=COURSE_PAL[0], lw=2, # green-teal "story" line
10        markersize=6)
11
12 ax.set_xlabel("time (hours)")
13 ax.set_ylabel("tpm")
14 ax.set_title("BRCA1 expression over 24h", color=INK)
15 ax.spines[["top", "right"]].set_visible(False)
```



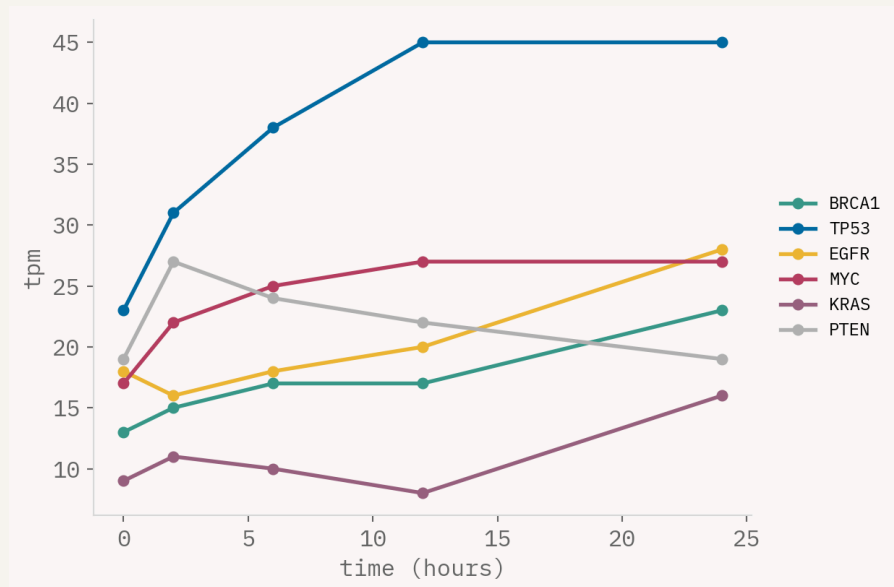
Time Series • Step 2 — All Six Genes

TIME SERIES • BUILD

2 • One line per gene, distinct on-palette colours

Loop the six genes and draw one line per gene. The course palette covers five; for the sixth we use a neutral grey — a deliberate de-emphasis, the gene that doesn't compete for attention.

```
1 tc = load_timecourse() # full long-for
2 genes = ["BRCA1", "TP53", "EGFR", "MYC", "KRAS", "PTEN"] # fixed order
3
4 # six colours: five course-palette + one neutral GREY for the last
5 palette = COURSE_PAL + [GREY] # length-6 col
6
7 fig, ax = plt.subplots(figsize=(6.6, 4.4)) # CREATE the c
8
9 for gene, colour in zip(genes, palette): # one line per
10     g = tc[tc["gene"] == gene].sort_values("time") # filter + ord
11     ax.plot(g["time"], g["tpm"], # CALL: trajec
12            marker="o", color=colour, lw=2,
13            markersize=5, label=gene)
14
15 ax.set_xlabel("time (hours)")
16 ax.set_ylabel("tpm")
17 ax.spines[["top", "right"]].set_visible(False)
18 ax.legend(loc="center left", bbox_to_anchor=(1.02, 0.5),
19         frameon=False, fontsize=9)
```



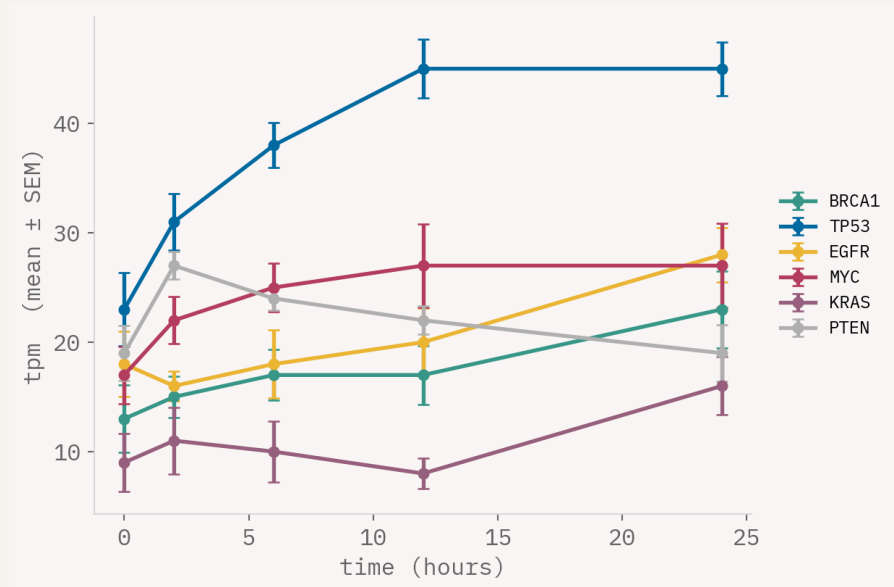
Time Series • Step 3 — Error Bars

TIME SERIES • BUILD

3 • `ax.errorbar` adds vertical whiskers at every point

Swap `ax.plot` for `ax.errorbar` and pass `yerr=g["sem"]`. The `capsize=3` gives the little horizontal caps that make whiskers readable. **Bracket access is mandatory** — `g.sem` is the method (slide 17).

```
1 tc = load_timecourse()
2 genes = ["BRCA1", "TP53", "EGFR", "MYC", "KRAS", "PTEN"]
3 palette = COURSE_PAL + [GREY]
4
5 fig, ax = plt.subplots(figsize=(6.6, 4.4))
6
7 for gene, colour in zip(genes, palette):
8     g = tc[tc["gene"] == gene].sort_values("time")
9
10    # CALL: line + whiskers in one call. yerr=g["sem"], NOT g.sem
11    ax.errorbar(g["time"], g["tpm"],
12               yerr=g["sem"],
13               marker="o", color=colour, lw=2,
14               markersize=5,
15               capsize=3,
16               label=gene)
17
18 ax.set_xlabel("time (hours)")
19 ax.set_ylabel("tpm (mean ± SEM)")
20 ax.spines[["top", "right"]].set_visible(False)
21 ax.legend(loc="center left", bbox_to_anchor=(1.02, 0.5),
22          frameon=False, fontsize=9)
```



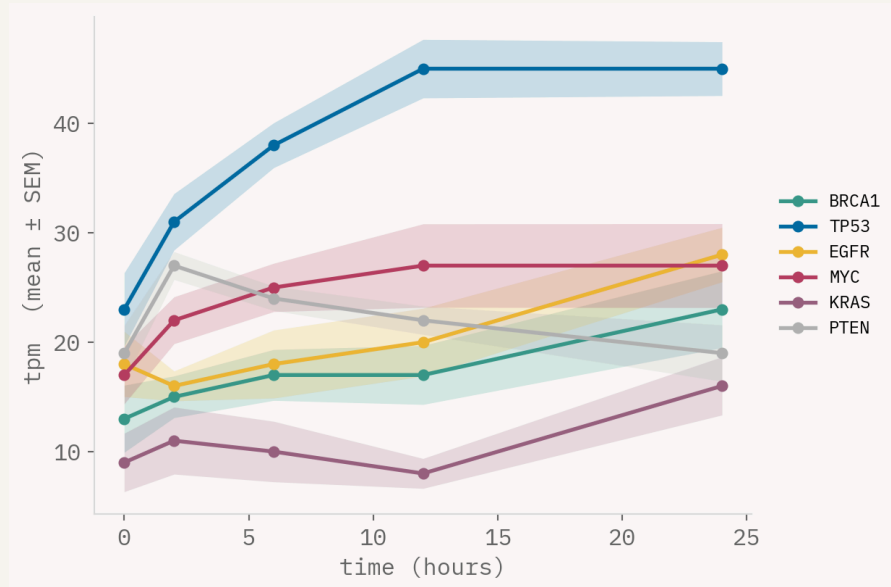
Time Series • Step 4 — Ribbons

TIME SERIES • BUILD

4 • `ax.fill_between` replaces whiskers with a translucent envelope

When lines are dense, whiskers stack and clutter. Swap them for a **ribbon**: `ax.fill_between(time, tpm - sem, tpm + sem, alpha=0.25)`. The trajectory stays sharp; the uncertainty becomes a soft halo around it.

```
1 tc = load_timecourse()
2 genes = ["BRCA1", "TP53", "EGFR", "MYC", "KRAS", "PTEN"]
3 palette = COURSE_PAL + [GREY]
4
5 fig, ax = plt.subplots(figsize=(6.6, 4.4))
6
7 for gene, colour in zip(genes, palette):
8     g = tc[tc["gene"] == gene].sort_values("time")
9
10    # CALL 1: the trajectory line, sharp on top
11    ax.plot(g["time"], g["tpm"],
12           color=colour, lw=2, marker="o", markersize=5, label=gene)
13
14    # CALL 2: the uncertainty ribbon, translucent below
15    ax.fill_between(g["time"],
16                  g["tpm"] - g["sem"],          # lower edge of the l
17                  g["tpm"] + g["sem"],          # upper edge of the l
18                  color=colour, alpha=0.20,    # see-through halo
19                  linewidth=0)
20
21 ax.set_xlabel("time (hours)")
22 ax.set_ylabel("tpm (mean ± SEM)")
```



Time Series • Step 5 — Small Multiples

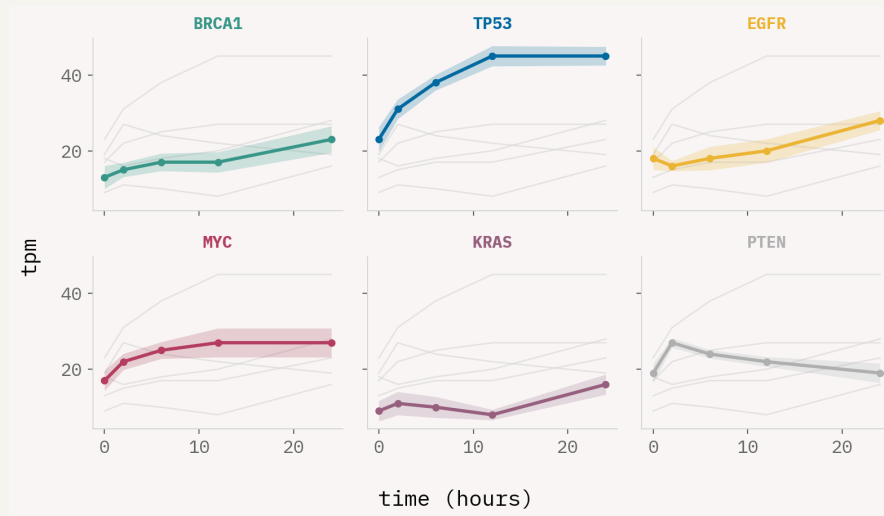
TIME SERIES • BUILD

5 • One panel per gene, faint grey context behind each focal trajectory

The killer plot. A 2×3 grid, **one gene per panel** in its colour with its SEM ribbon, and *every other gene drawn faintly in grey behind it* so each panel has full context.

`sharex=True, sharey=True` keeps the axes comparable across panels.

```
1 tc = load_timecourse()
2 genes = ["BRCA1", "TP53", "EGFR", "MYC", "KRAS", "PTEN"]
3 palette = COURSE_PAL + [GREY]
4
5 # CREATE: 2 rows x 3 cols, shared axes for easy across-panel comparison
6 fig, axes = plt.subplots(2, 3, figsize=(8.4, 4.8),
7                           sharex=True, sharey=True)
8 axes = axes.flatten() # 6 axes, 6 panels
9
10 for ax, gene, colour in zip(axes, genes, palette):
11     # context: every other gene as a faint grey line behind
12     for other in genes:
13         if other == gene: # skip the focal gene
14             continue
15         o = tc[tc["gene"] == other].sort_values("time")
16         ax.plot(o["time"], o["tpm"],
17               color=LINE, lw=1, alpha=0.6, zorder=1) # de-emphasize
18
19     # focal gene: full colour, ribbon, marker
20     g = tc[tc["gene"] == gene].sort_values("time")
21     ax.plot(g["time"], g["tpm"], color=colour, lw=2.2,
22           marker="o", markersize=4, zorder=2) # on top of context
```



Time Series • Seaborn Variant

TIME SERIES • ERGONOMIC

One `sns.lineplot` call replaces the loop

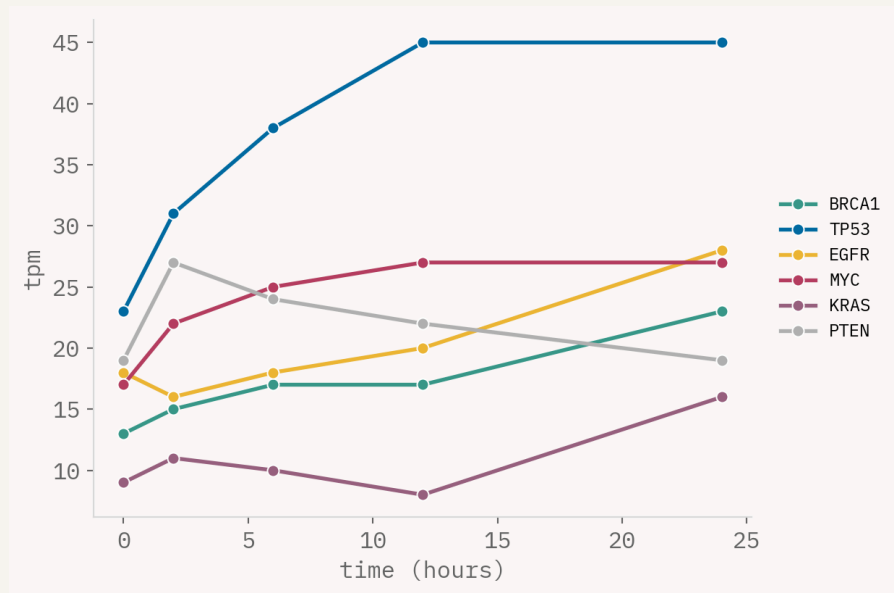
Seaborn understands long-form data natively. Pass the whole `tc` table, name the columns, and use `hue="gene"` — seaborn does the per-gene split, the colour cycling, and the standard-error ribbon (`errorbar="se"`) in one call. The cost: less control over each line's styling.

```
1 tc = load_timecourse()
2
3 # CALL: one line per `gene`, ribbon from `errorbar="se"`
4 ax = sns.lineplot(
5     data=tc, x="time", y="tpm",
6     hue="gene",                                # split by gene -> 6 c
7     hue_order=["BRCA1", "TP53", "EGFR", "MYC", "KRAS", "PTEN"],
8     palette=COURSE_PAL + [GREY],               # the same 6-colour li
9     errorbar="se",                             # ribbon = ± standard
10    marker="o", linewidth=2)
11
12 ax.set_xlabel("time (hours)"); ax.set_ylabel("tpm")
13 ax.spines[["top", "right"]].set_visible(False)
14 ax.legend(loc="center left", bbox_to_anchor=(1.02, 0.5),
15          frameon=False, fontsize=9, title="")
```

Trade-off: seaborn collapses the loop and adds a ribbon for free — perfect for **exploration**. matplotlib's explicit loop wins when you need **bespoke styling per line** (one

bold, the rest dimmed; mixed dashes; small-multiples).

Use seaborn to *see*, matplotlib to *publish*.



Time Series • Takeaway

TIME SERIES • RECAP

Trajectory plus uncertainty, scaled to your data

THE RECIPE

1. **Lines tell direction**, error bars or ribbons tell **confidence**. 2. **Error bars for few points** (≤ 6 timepoints); **ribbons for many**. 3. **Small multiples** isolate each story when lines overlap — keep faint grey context behind each panel. 4. **Always use bracket access** for `df["sem"]` — the dot-attribute collides with `DataFrame.sem()` and silently breaks.

Direction + confidence + isolation: the three knobs of every time-series figure. Pick the one your audience needs most for *this* plot.

Multiple Sequence Alignment

COMPARATIVE SEQUENCES

What Is an MSA?

MSA • THEORY

N sequences, one column at a time

A **multiple sequence alignment (MSA)** stacks N related sequences so that **homologous positions sit in the same column**. Reading top-to-bottom in one column tells you how that letter varies across species, samples, or strains. Reading left-to-right is one full sequence.

Three things the eye looks for:

- **Conserved columns** — every row carries the same letter. These mark functionally important sites (active-site residues, splice motifs, transcription-factor footprints).
- **Gap columns** (-) — an **insertion** in one lineage or a **deletion** in another. Gaps interrupt the grid like vertical white stripes.
- **Mismatched columns** — divergence. Useful for phylogenetic distance, but only after the alignment is trustworthy.

Colour encodes chemistry: the Clustal scheme groups hydrophobic, polar, acidic, and basic residues so families pop. For DNA we just colour by **identity** (A, C, G, T) — four hues is enough.

When to use it: phylogeny, conserved-domain hunting, primer design, regulatory motif discovery,

MSA • The Data

MSA • DATA PEEK

Synthesise 8 sequences × 60 columns, low mutation rate + one indel

No real loader: we **build a toy alignment in numpy**. Start from one random ACGT base sequence, derive seven mutants with ~5 % per-base substitutions, then punch a small deletion (-) into one sequence at columns 22–26. That gives a realistic-looking conservation pattern with one obvious gap.

```
1 # 60-column base sequence drawn uniformly from ACGT
2 n_cols = 60 # alignment width
3 n_rows = 8 # number of sequences
4 bases = np.array(list("ACGT")) # the alphabet
5
6 # seq 0 is the ancestor; seqs 1..7 inherit then mutate
7 base = rng.choice(bases, size=n_cols) # one random ACGT row
8 aln = np.tile(base, (n_rows, 1)) # 8 identical copies
9
10 # at each position in each non-ancestor row, mutate with p = 5 %
11 mutate = rng.random((n_rows, n_cols)) < 0.05 # boolean mask
12 mutate[0] = False # row 0 stays as ancestor
13 aln[mutate] = rng.choice(bases, size=mutate.sum()) # swap in a new base
14
15 # introduce a clean 5-column deletion in sequence 5 (cols 22..26)
16 aln[5, 22:27] = "-" # the indel block
17
18 print(aln.shape) # (8, 60)
19 print("\n".join("".join(row) for row in aln)) # show the alignment text
```

(8, 60)

AATCGGGACACTGAGATTTGTCAGCGTCTACTAGCGTTTGTTAACGTCTCTGGAGTTTAA
AATCGGGACACTGAGTTTTTTCAGCGTCTATTAGCGTTTGTTAACGTCTCTGAAGTTTAA

```
AATCGGGACACTGAGATTTGTCAGCGTCTACTAGCGTTTGTTAACGTCTCTGGAGTTTAA
AATCGTGACACTGAGATTTGTCAGCGTCTACTAGCCGTTGTTAACGTCTCTGGAGTTTAA
AATCGGGACACTGAGATTAGTCAGCGTCTAGTACCGTTTGGTAACGTGTCTGGAGTTTTT
AATCGGGACACTGAGATTAGTC-----TTACTAGCGTTTGTTAACGTCTCTGGAGTTTAA
AATCGGGACACTGAGATTTGTCAGCGTCTACTAGCGTTTGTTAACGTCTCTGGAGTTTAA
AATCGGGACACTGAGATTTGTCAGCGTCTACTAGCGTGTGTTAACGTCTCTGAAGTTTAA
```


MSA • Step 1 — Coloured Letter Grid

MSA • BUILD

1 • Draw each base as text at (col, row)

The simplest renderer: walk the array and place every letter with `ax.text(col, row, letter, color=...)`.

Matplotlib uses **data coordinates**, so `(col, row)` puts column 0 at the left edge and row 0 at the top — but matplotlib's y-axis grows **upward** by default, so we **invert the y-axis** to read top-to-bottom like a text file.

```
1 # rebuild the alignment so the slide is self-contained
2 n_cols, n_rows = 60, 8 # alignment shape
3 bases = np.array(list("ACGT")) # DNA alphabet
4
5 base = rng.choice(bases, size=n_cols) # ancestor row
6 aln = np.tile(base, (n_rows, 1)) # 8 copies
7 mutate = rng.random((n_rows, n_cols)) < 0.05 # 5 % mutation rate
8 mutate[0] = False # keep ancestor copy
9 aln[mutate] = rng.choice(bases, size=mutate.sum()) # apply mutations
10 aln[5, 22:27] = "-" # one indel block
11
12 # colour by base identity — Economist palette, GREY for gaps
13 col_map = {"A": GREEN, "C": BLUE, "G": AMBER, "T": RED, "-": GREY}
14
15 fig, ax = plt.subplots(figsize=(9.5, 2.6)) # CREATE wide canvas
16 for r in range(n_rows): # one row of sequence
17     for c in range(n_cols): # one column position
18         letter = aln[r, c] # the base at (r, c)
19         ax.text(c, r, letter, ha="center", va="center", # CALL: draw letter
20               color=col_map[letter], fontsize=9, fontweight="bold")
21
22 ax.set_xlim(-0.5, n_cols - 0.5) # column range
```

```
seq0 - GTTCACCTCCATCCAGCTTTGAGTTCAAAGGCTAGTCGACCGCTCAGACTTTCGATCATCAG
seq1 - GTTCACCTCCATCCAGCTTTGAGTTCAAAGGCTAGTCGACCGCTCAGACTTTCGATCATCAG
seq2 - GTTCACCTCCATCCAGCTTTGAGTTCAAAGGCTAGTCGACCGCTCAGACTTTCGATCATCAG
seq3 - GTTCACCTCCATCCAGCTTTGAGTTCAAAGGCTAGTCGACCGCTCAGACTTTCGATCATCAG
seq4 - GTTCACCTCCATCCAGCTTTGAGTTCAAAGGCTAGTCGACCGCTCAGACTTTCGATCATCAG
seq5 - GTTCACCTCCATCCAGCTTTGAGTTCAAAGGCTAGTCGACCGCTCAGACTTTCGATCATCAG
seq6 - GTTCACCTCCATCCAGCTTTGAGTTCAAAGGCTAGTCGACCGCTCAGACTTTCGATCATCAG
seq7 - GTTCACCTCCATCCAGCTTTGAGTTCAAAGGCTAGTCGACCGCTCAGACTTTCGATCATCAG
```

MSA • Step 2 — Heatmap Encoding

MSA • BUILD

2 • Paint cells, overlay letters in white

The letter-grid is friendly but **doesn't scale past ~30 rows**. Replace it with a coloured heatmap: encode each base as an integer (A=0, C=1, G=2, T=3, -=4), feed that to `ax.imshow` with a `ListedColormap` of the five palette colours, then overlay each letter in white on top. The same picture, now scalable to hundreds of sequences.

```
1 from matplotlib.colors import ListedColormap
2
3 # rebuild alignment
4 n_cols, n_rows = 60, 8
5 bases = np.array(list("ACGT"))
6 base = rng.choice(bases, size=n_cols); aln = np.tile(base, (n_rows, 1))
7 mutate = rng.random((n_rows, n_cols)) < 0.05; mutate[0] = False
8 aln[mutate] = rng.choice(bases, size=mutate.sum()); aln[5, 22:27] = "-"
9
10 # encode each letter as an integer in [0..4]
11 code = {"A": 0, "C": 1, "G": 2, "T": 3, "-": 4} # letter -> index
12 aln_int = np.vectorize(code.get)(aln) # int array, same
13 cmap = ListedColormap([GREEN, BLUE, AMBER, RED, "#FFFFFF"]) # one colour
14
15 fig, ax = plt.subplots(figsize=(9.5, 2.6)) # CREATE canvas
16 ax.imshow(aln_int, cmap=cmap, aspect="auto", # CALL: paint cell
17           interpolation="nearest", vmin=0, vmax=4)
18
19 # overlay each letter in white so the identity is still legible
20 for r in range(n_rows):
21     for c in range(n_cols):
22         letter = aln[r, c]
```



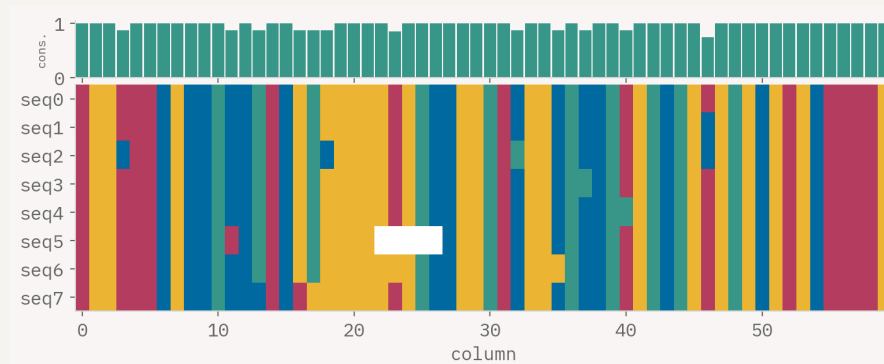
MSA • Step 3 — Conservation Track

MSA • BUILD

3 • Bar above the grid, one bar per column

The canonical MSA viewer layout is **two stacked panels** **sharing the x-axis**: a tall bottom panel with the alignment, a short top panel with a **conservation bar** per column. Compute conservation as the **fraction of rows showing the most-common non-gap base** — 1.0 means every sequence agrees, 0.25 means random.

```
1 from matplotlib.colors import ListedColormap
2
3 n_cols, n_rows = 60, 8
4 bases = np.array(list("ACGT"))
5 base = rng.choice(bases, size=n_cols); aln = np.tile(base, (n_rows, 1))
6 mutate = rng.random((n_rows, n_cols)) < 0.05; mutate[0] = False
7 aln[mutate] = rng.choice(bases, size=mutate.sum()); aln[5, 22:27] = "-"
8
9 # conservation = fraction of most-common non-gap letter per column
10 def col_conservation(col):
11     non_gap = col[col != "-"]           # drop gaps before
12     if len(non_gap) == 0:               # all-gap column
13         return 0.0
14     _, counts = np.unique(non_gap, return_counts=True) # tally each base
15     return counts.max() / len(non_gap)         # majority fraction
16 conservation = np.array([col_conservation(aln[:, c]) for c in range(n_cols)]
17
18 # CREATE two stacked panels — short bar track on top, alignment below
19 fig, (axc, axa) = plt.subplots(        # axc = conservat
20     2, 1, figsize=(9.5, 3.4), sharex=True,
21     gridspec_kw=dict(height_ratios=[1, 4], hspace=0.05))
22
```



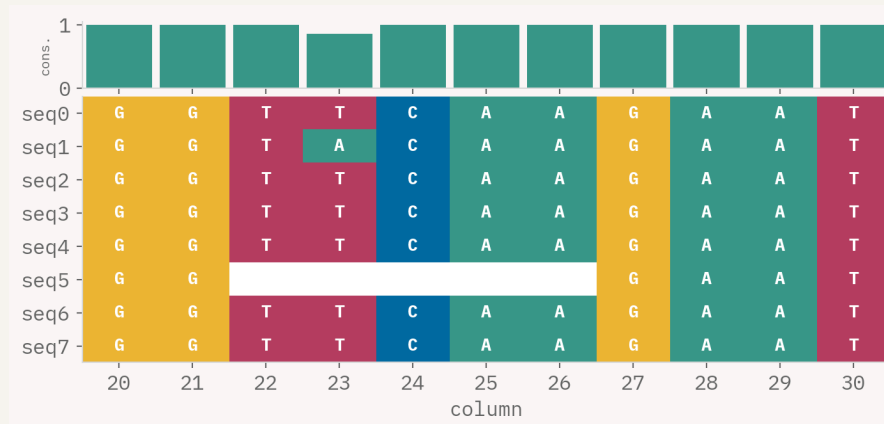
MSA • Step 4 — Zoom to the Indel

MSA • BUILD

4 • Crop to columns 20–30; the gap is the story

At full width every column is one pixel wide and the eye washes out. **Zoom** with `ax.set_xlim(20, 30)` to the indel region — now the gap block (cols 22–26 in `seq5`) reads as five empty cells, and the surrounding conservation is clear column by column. Working MSAs always combine an overview with a zoom.

```
1 from matplotlib.colors import ListedColormap
2
3 n_cols, n_rows = 60, 8
4 bases = np.array(list("ACGT"))
5 base = rng.choice(bases, size=n_cols); aln = np.tile(base, (n_rows, 1))
6 mutate = rng.random((n_rows, n_cols)) < 0.05; mutate[0] = False
7 aln[mutate] = rng.choice(bases, size=mutate.sum()); aln[5, 22:27] = "-"
8
9 def col_conservation(col):
10     non_gap = col[col != "-"]
11     if len(non_gap) == 0: return 0.0
12     _, counts = np.unique(non_gap, return_counts=True)
13     return counts.max() / len(non_gap)
14 conservation = np.array([col_conservation(aln[:, c]) for c in range(n_cols)]
15
16 fig, (axc, axa) = plt.subplots(
17     2, 1, figsize=(8.5, 3.6), sharex=True,
18     gridspec_kw=dict(height_ratios=[1, 4], hspace=0.05))
19
20 axc.bar(np.arange(n_cols), conservation, color=GREEN, width=0.9)
21 axc.set_ylim(0, 1.05); axc.set_yticks([0, 1])
22 axc.set_ylabel("cons.", fontsize=8)
```



MSA • Production Tools

MSA • ECOSYSTEM

Biopython for I/O, pyMSAviz for paper-ready output

For real work, don't roll your own — two libraries cover it:

Biopython reads/writes every common alignment format (FASTA, Clustal, Stockholm, PHYLIP) and gives you a numpy-like view:

```
1 from Bio import AlignIO
2 aln = AlignIO.read("globins.aln", "clustal")
3 for record in aln:
4     print(record.id, record.seq[:60])
```

The matplotlib build you just saw is the **mental model**; these libraries do the same thing with more polish and more formats.

pyMSAviz renders Clustal-coloured, paper-ready figures with conservation tracks, consensus rows, and reasonable defaults:

```
1 from pymsaviz import MsaViz
2 mv = MsaViz("globins.aln", color_scheme="Clustal",
3           wrap_length=60, show_consensus=True)
4 mv.savefig("globins_msa.png")
```

MSA • Takeaway

MSA • RECAP

What the eye is reading

THE RECIPE

1. **Conservation, identity, gaps** — three things the reader scans for in every column. 2. **Colour encodes chemistry** (Clustal) or **identity** (A/C/G/T). Four-to-five hues is enough; gaps are grey or white. 3. **Conservation track on top** of the heatmap turns the eye to interesting columns automatically. 4. **Always zoom**. A full-width MSA is a dense mosaic; the story lives in a 10–20 column window.

Manhattan Plots

GWAS · GENOME-WIDE SIGNIFICANCE

What Is a Manhattan Plot?

MANHATTAN • THEORY

One dot per SNP, the whole genome on the x-axis

A Manhattan plot is the canonical view of a **genome-wide association study (GWAS)**. Each dot is one variant.

The **x-axis is genomic position** — every chromosome laid end-to-end — and the **y-axis is the $-\log_{10}$ of the association p-value**. So *higher = stronger evidence* that the variant is linked to the trait.

The shape that gives it its name: most SNPs are background noise, a flat low cloud — but real signals rise into thin **skyscrapers** that look like the New York skyline. The eye reads “significance peaks above the horizon line”.

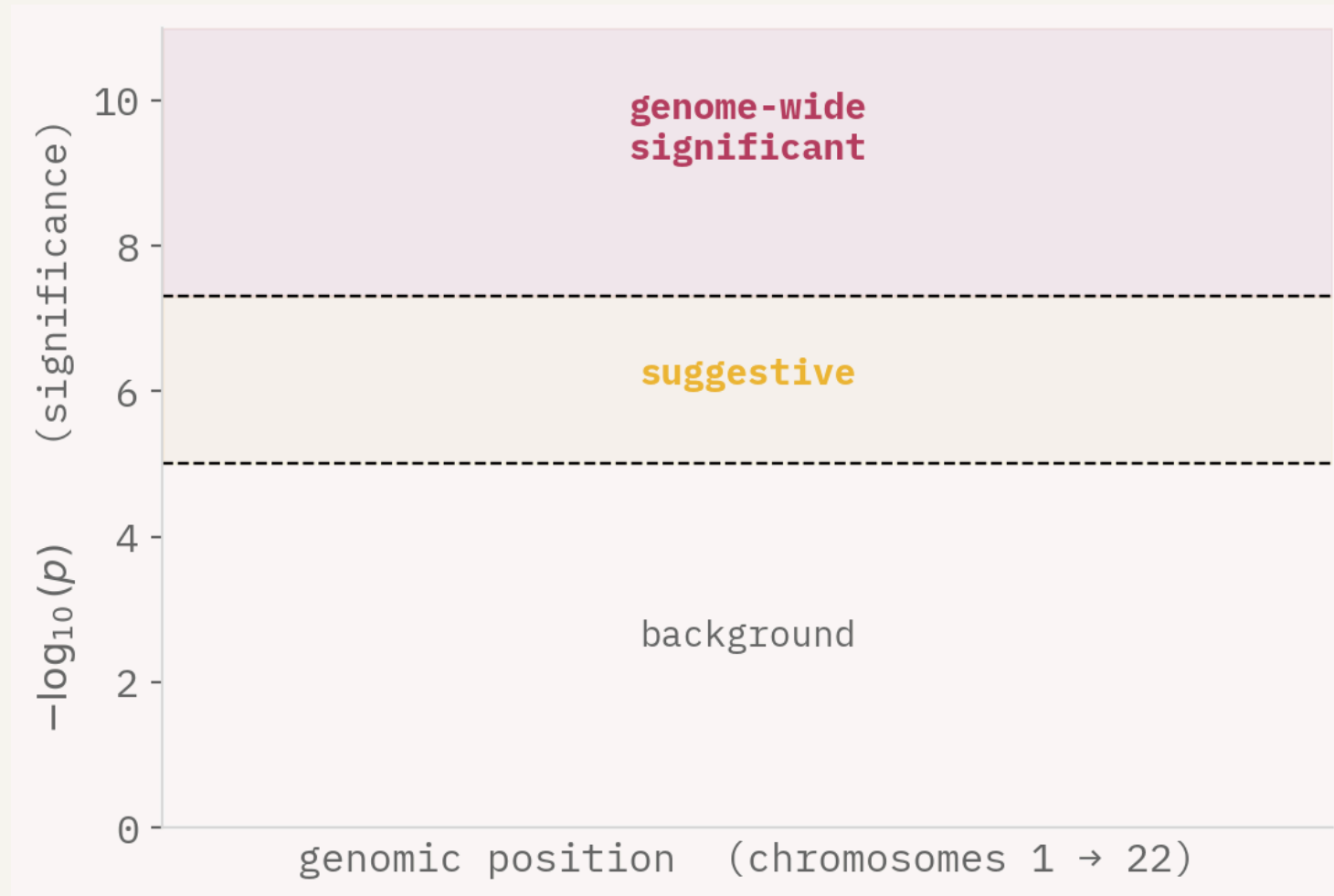
Two horizontal cutoffs:

- **Suggestive line:** $p = 1e-5$ (often shown as $-\log_{10}(p) = 5$)
- **Genome-wide significance line:** $p = 5e-8$ ($-\log_{10}(p) \approx 7.3$) — the canonical Bonferroni-corrected threshold for ~1 M independent tests in the human genome.

Anatomy, Before Any Data

MANHATTAN • THEORY

Three regions before we even load a file



Manhattan • The Data

MANHATTAN • DATA PEEK

One row per SNP — chrom, pos, p-value, rsID

`load_gwas("height")` returns a tidy table: 13 475 SNPs across 22 chromosomes, with raw p-values. The simulation seeds a few real-looking peaks so the build has something to point at.

```
1 gwas = load_gwas("height")      # chrom, pos, pval, snp
2 gwas.head(5)
```

	chrom	pos	pval	snp
0	1	562879	0.197842	rs1217797
1	1	802716	0.454291	rs3152883
2	1	933561	0.750287	rs1966161
3	1	1294717	0.707315	rs2836508
4	1	1316327	0.553459	rs6514392

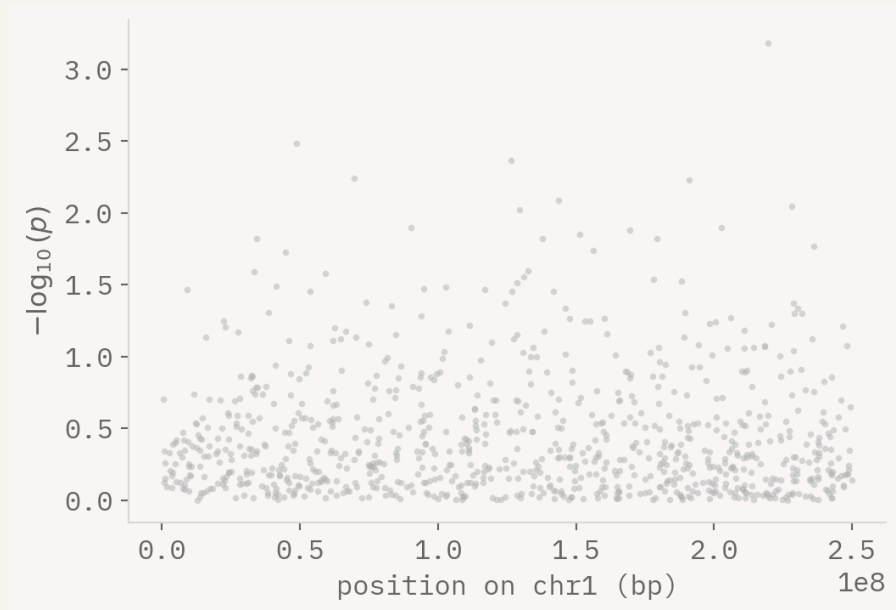
Manhattan • Step 1 — One Chromosome First

MANHATTAN • BUILD

1 • The cloud, on chromosome 1 only

Before we tile 22 chromosomes, build the **idea** on one.
Scatter SNP position vs $-\log_{10}(p)$. The shape is already there: a flat cloud with the occasional spike.

```
1 gwas = load_gwas("height")           # the whole-genome table
2
3 chr1 = gwas[gwas["chrom"] == 1].copy() # one chromosome only
4 chr1["nlog10p"] = -np.log10(chr1["pval"]) # significance axis
5
6 fig, ax = plt.subplots(figsize=(6, 4)) # CREATE the canvas
7 ax.scatter(chr1["pos"], chr1["nlog10p"], # CALL: x = bp, y = -log10(
8           s=8, color=GREY, alpha=0.5, linewidths=0)
9
10 ax.set_xlabel("position on chr1 (bp)")
11 ax.set_ylabel(r"$-\log_{10}(p)$")
12 ax.spines[["top", "right"]].set_visible(False)
```



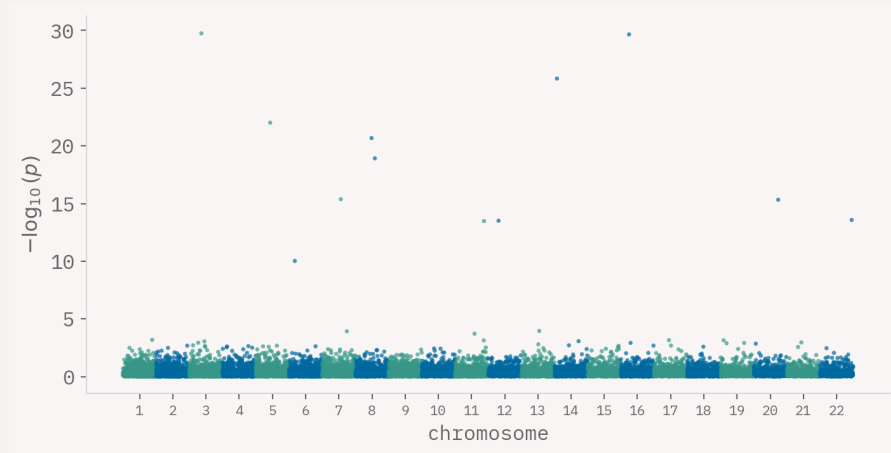
Manhattan • Step 2 — All 22, Alternating Colours

MANHATTAN • BUILD

2 • Lay every chromosome end-to-end

The trick: replace per-chromosome `pos` with a **cumulative** coordinate so chr2 starts where chr1 ends. Alternate two colours so the eye can tell chromosomes apart without a heavy x-axis.

```
1 gwas = load_gwas("height").sort_values(["chrom", "pos"]).copy()
2 gwas["nlog10p"] = -np.log10(gwas["pval"])
3
4 # 1. find each chromosome's length, then offset cumulatively
5 chrom_len = gwas.groupby("chrom")["pos"].max()
6 offsets = chrom_len.cumsum().shift(fill_value=0) # chr1→0, chr2→len1
7 gwas["cum_pos"] = gwas["pos"] + gwas["chrom"].map(offsets)
8
9 # 2. one colour per parity (Economist BLUE / GREEN), painless cycler
10 palette = {True: BLUE, False: GREEN}
11 gwas["color"] = (gwas["chrom"] % 2 == 0).map(palette)
12
13 # 3. scatter once with the precomputed colour column
14 fig, ax = plt.subplots(figsize=(8.5, 4))
15 ax.scatter(gwas["cum_pos"], gwas["nlog10p"],
16           c=gwas["color"], s=6, alpha=0.7, linewidths=0)
17
18 # 4. x-ticks at each chromosome's mid-point, not every megabase
19 tick_pos = offsets + chrom_len / 2
20 ax.set_xticks(tick_pos); ax.set_xticklabels(chrom_len.index, fontsize=8)
21 ax.set_xlabel("chromosome"); ax.set_ylabel(r"$-\log_{10}(p)$")
22 ax.spines[["top", "right"]] set visible(False)
```



Manhattan • Step 3 — The Significance Lines

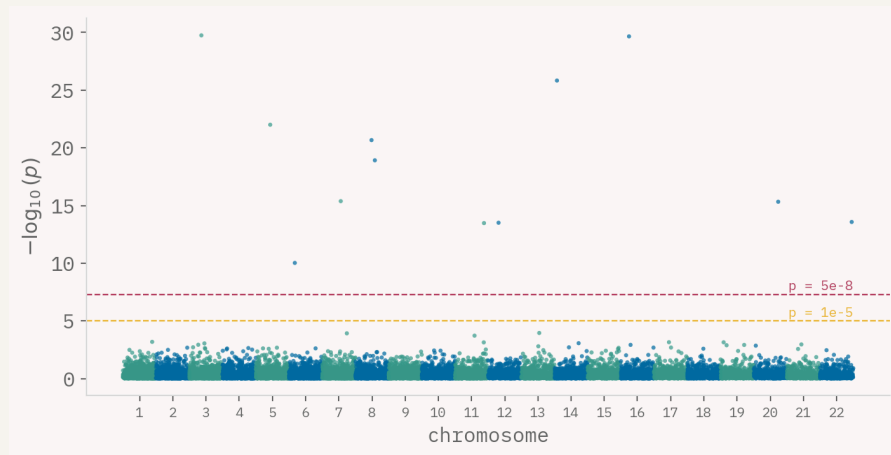
MANHATTAN • BUILD

3 • Draw the thresholds the field agrees on

Two dashed horizontals turn the cloud into a test:

suggestive ($p = 1e-5$) and genome-wide ($p = 5e-8$). Anything above the upper line is the headline of the paper.

```
1 gwas = load_gwas("height").sort_values(["chrom", "pos"]).copy()
2 gwas["nlog10p"] = -np.log10(gwas["pval"])
3
4 chrom_len = gwas.groupby("chrom")["pos"].max()
5 offsets = chrom_len.cumsum().shift(fill_value=0)
6 gwas["cum_pos"] = gwas["pos"] + gwas["chrom"].map(offsets)
7
8 palette = {True: BLUE, False: GREEN}
9 gwas["color"] = (gwas["chrom"] % 2 == 0).map(palette)
10
11 fig, ax = plt.subplots(figsize=(8.5, 4))
12 ax.scatter(gwas["cum_pos"], gwas["nlog10p"],
13           c=gwas["color"], s=6, alpha=0.7, linewidths=0)
14
15 tick_pos = offsets + chrom_len / 2
16 ax.set_xticks(tick_pos); ax.set_xticklabels(chrom_len.index, fontsize=8)
17 ax.set_xlabel("chromosome"); ax.set_ylabel(r"$-\log_{10}(p)$")
18 ax.spines[["top", "right"]].set_visible(False)
19
20 # 5. the thresholds — dashed, annotated on the right edge
21 ax.axhline(-np.log10(5e-8), color=RED, lw=.9, ls="--") # genome-wide
22 ax.axhline(-np.log10(1e-5), color=AMBER, lw=.9, ls="--") # suggestive
```



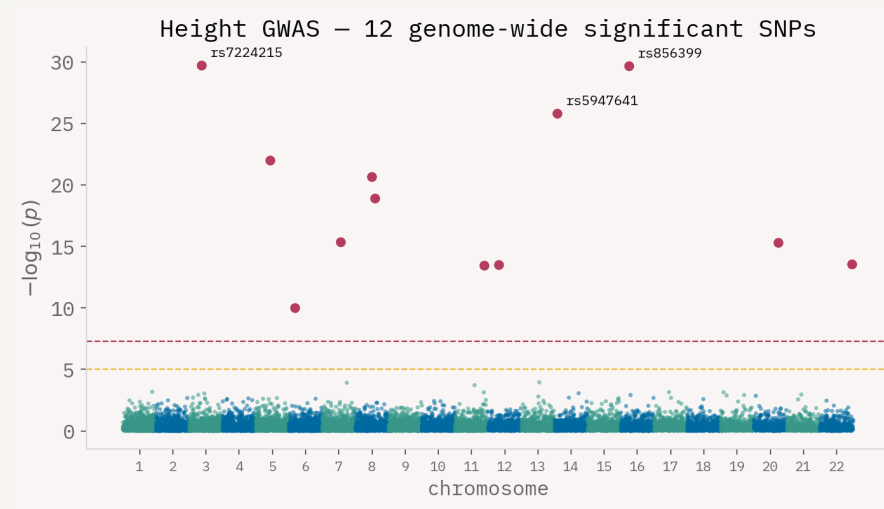
Manhattan • Step 4 — Highlight & Label the Hits

MANHATTAN • BUILD

4 • Spotlight the SNPs that crossed the line

Genome-wide significant hits get a **filled red dot on top of the cloud** and an rsID label. The audience finally sees what the paper claims.

```
1 gwas = load_gwas("height").sort_values(["chrom", "pos"]).copy()
2 gwas["nlog10p"] = -np.log10(gwas["pval"])
3
4 chrom_len = gwas.groupby("chrom")["pos"].max()
5 offsets = chrom_len.cumsum().shift(fill_value=0)
6 gwas["cum_pos"] = gwas["pos"] + gwas["chrom"].map(offsets)
7
8 palette = {True: BLUE, False: GREEN}
9 gwas["color"] = (gwas["chrom"] % 2 == 0).map(palette)
10
11 fig, ax = plt.subplots(figsize=(8.5, 4.3))
12 ax.scatter(gwas["cum_pos"], gwas["nlog10p"],
13           c=gwas["color"], s=6, alpha=0.55, linewidths=0)
14
15 ax.axhline(-np.log10(5e-8), color=RED, lw=.9, ls="--")
16 ax.axhline(-np.log10(1e-5), color=AMBER, lw=.9, ls="--")
17
18 tick_pos = offsets + chrom_len / 2
19 ax.set_xticks(tick_pos); ax.set_xticklabels(chrom_len.index, fontsize=8)
20
21 # 6. spotlight — the hits as bigger red dots, on top (high zorder)
22 hits = gwas[gwas["nlog10p"] > 5.99]
```



Manhattan • Plotly Variant

MANHATTAN • INTERACTIVE

Hover for the SNP ID — same plot, browser-native interactivity

For dashboards and exploratory review, plotly gives every dot a tooltip. Same logic as the matplotlib version — cumulative x, parity colour, threshold line — but `hover_data` exposes chrom / pos / rsID.

```
1 import plotly.express as px
2
3 # reuse the prepared frame (cum_pos, nlog10p, color)
4 gwas = load_gwas("height").sort_values(["chrom", "pos"]).copy()
5 gwas["nlog10p"] = -np.log10(gwas["pval"])
6 chrom_len = gwas.groupby("chrom")["pos"].max()
7 offsets = chrom_len.cumsum().shift(fill_value=0)
8 gwas["cum_pos"] = gwas["pos"] + gwas["chrom"].map(offsets)
9 gwas["chrom_parity"] = (gwas["chrom"] % 2 == 0).astype(str)
10
11 # px.scatter handles colour grouping + hover for us
12 fig = px.scatter(
13     gwas, x="cum_pos", y="nlog10p",
14     color="chrom_parity",
15     color_discrete_map={"True": BLUE, "False": GREEN},
16     hover_data=["chrom", "pos", "snp", "pval"],
17     labels={"cum_pos": "genomic position", "nlog10p": "-log10(p)"})
18
19 fig.add_hline(y=-np.log10(5e-8), line_dash="dash", line_color=RED)
20 fig.add_hline(y=-np.log10(1e-5), line_dash="dash", line_color=AMBER)
21 fig.update_traces(marker=dict(size=4, opacity=0.6))
22 fig.update_layout(showlegend=False, height=420)
```

WebGL is not supported by
your browser - visit
<https://get.webgl.org> for
more info

Manhattan • Takeaway

MANHATTAN • RECAP

Four ideas, every time

THE RECIPE

1. **Cumulative position** so chromosomes lay end-to-end. 2. **Two-colour parity** so neighbours are visually separable without ticks. 3. **Two threshold lines** — suggestive ($1e-5$) and genome-wide ($5e-8$). 4. **Highlight + label** the hits; the cloud is context, the spikes are the story.

The same four ideas drive most genome-scale dot plots: methylation Mannhattans, peak-significance plots, mutation rate scans. Once you've built one, you've built them all.

Genome Statistics

COVERAGE · PEAKS · GENE MODELS

Three Questions, Three Charts

GENOME STATS • THEORY

What is the chart actually answering?

Genome-stats figures crowd into papers and dashboards, but each one is really asking **one** of three questions:

- **Is my sample any good?** Coverage and callable-fraction QC: per-sample bar charts, dumbbells, ridge plots. The chart fails if the reader can't spot the dropouts at a glance.
- **Where in the genome is the signal?** Peak tracks for ChIP-seq, ATAC-seq, methylation. The x-axis is **genomic coordinate**; the y-axis is signal height. Multiple marks stack vertically with a **shared x-axis**.
- **What gene is there?** A gene-model track of rectangles with strand-aware arrowheads, drawn under the signal panels so the reader can connect a peak to a feature.

A paper-ready ChIP-seq screenshot is just these three answers, stacked, sharing one x-axis. We'll build it.

Genome Stats • The QC Data

COVERAGE • DATA PEEK

One row per sample — raw vs callable rate, with the loss

`load_qc()` is twelve sequencing libraries with two metrics:

- `raw` — fraction of reads that map at all (alignment rate).
- `callable` — fraction of reads that survive duplicate removal, MAPQ filtering, and duplicate-clipping (the bases you can actually call variants from).
- `loss` — `raw - callable`. **Large loss = a problem sample:** high duplication, contamination, or PCR over-amplification.

```
1 qc = load_qc()           # sample, raw, callable, loss
2 qc.head(6)
```

	sample	raw	callable	loss
0	S01	92.7	88.2	4.5
1	S02	94.1	68.1	26.0
2	S03	92.7	88.1	4.6
3	S04	95.0	92.9	2.1
4	S05	93.7	89.2	4.5
5	S06	93.4	89.3	4.1

Coverage · Step 1 — One Sample, Two Bars

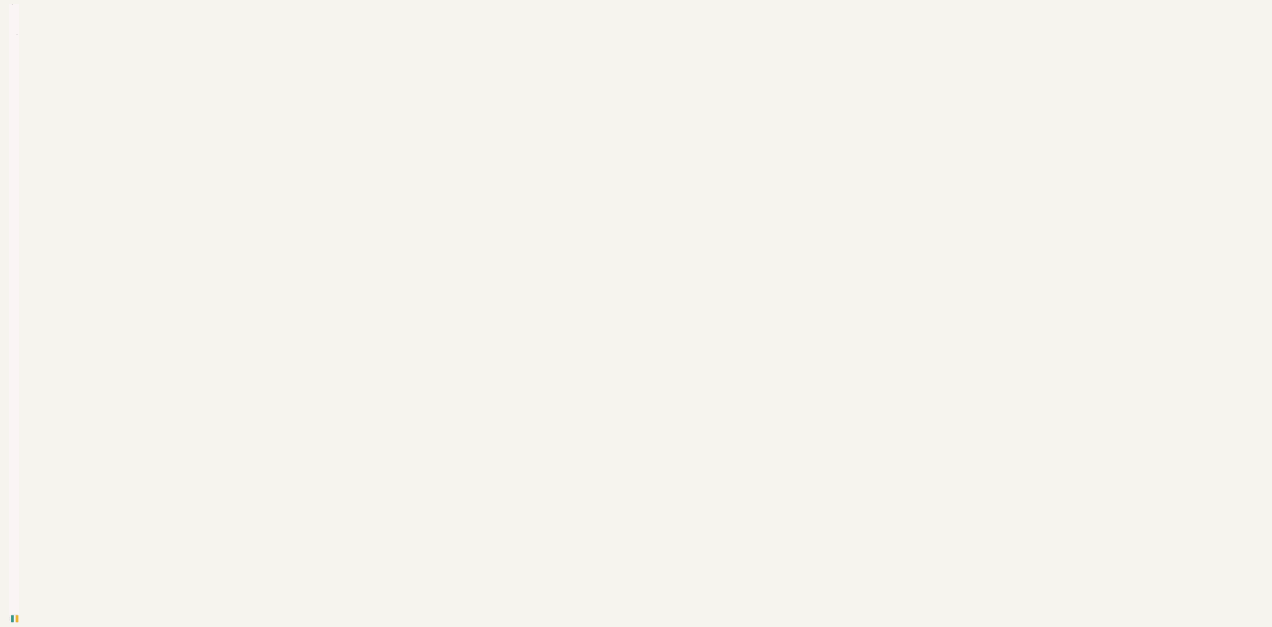
COVERAGE · BUILD

1 · The simplest possible view

Start with a single sample to fix the colour grammar:

GREEN for `raw` (what mapped), **AMBER** for `callable` (what survived filters). Side-by-side bars make the gap visible.

```
1 qc = load_qc()                # 12 samples
2 one = qc.iloc[0]              # the first sample
3
4 vals = [one["raw"], one["callable"]] # two heights
5 labels = ["raw", "callable"]        # two bar names
6 colors = [GREEN, AMBER]            # palette grammar
7
8 fig, ax = plt.subplots(figsize=(4.4, 4.2)) # CREATE compact can
9 ax.bar(labels, vals, color=colors, width=0.6) # CALL: side-by-side
10
11 # annotate each bar with its percentage so the gap is unambiguous
12 for i, v in enumerate(vals):
13     ax.text(i, v + 0.01, f"{v:.0%}", ha="center", va="bottom",
14             color=INK, fontweight="bold", fontsize=11)
15
16 ax.set_ylim(0, 1.05)           # fraction axis
17 ax.set_ylabel("fraction of reads")
18 ax.set_title(f"{one['sample']} — raw vs callable", color=INK)
19 ax.spines[["top", "right"]].set_visible(False)
```



Coverage · Step 2 — Dumbbell Across All Samples

COVERAGE · BUILD

2 · One lane per sample, sorted by loss

Twelve side-by-side bar pairs is noisy. The canonical view is a **dumbbell**: each sample is one **horizontal lane** with a thin grey line spanning raw → callable, a **GREEN dot at raw**, and an **AMBER dot at callable**. Sort samples by **loss** (biggest first) and the worst offenders rise to the top.

```
1 qc = load_qc().sort_values("loss", ascending=False)      # worst loss on top
2 qc = qc.reset_index(drop=True)                          # 0..N for the y-axis
3
4 fig, ax = plt.subplots(figsize=(7.2, 4.6))              # CREATE wide-ish
5
6 # one thin grey line per sample, from raw down to callable
7 ax.hlines(y=qc.index, xmin=qc["callable"], xmax=qc["raw"],
8           color=LINE, lw=2, zorder=1)                   # the "bar" of the
9
10 # GREEN dot = where we started (raw); AMBER dot = what survived (callable,
11 ax.scatter(qc["raw"], qc.index, color=GREEN, s=80, zorder=3, label="raw")
12 ax.scatter(qc["callable"], qc.index, color=AMBER, s=80, zorder=3, label="callable")
13
14 # y-axis = sample label, top-to-bottom = worst loss to best
15 ax.set_yticks(qc.index)
16 ax.set_yticklabels(qc["sample"], fontsize=8)
17 ax.invert_yaxis()                                       # biggest loss at top
18 ax.set_xlim(0.4, 1.0)                                 # tighten the lens
19 ax.set_xlabel("fraction of reads")
20 ax.legend(loc="lower right", frameon=False, fontsize=9)
21
22 # annotate loss in % at the right edge of each lane
```

Genome Stats • The Peak Data

PEAKS + GENES • DATA PEEK

Two tidy tables — signal tracks and gene annotations

`load_peaks()` is **one row per genomic position** with three signal columns. `load_peak_genes()` is **one row per gene** with start/end/strand.

```
1 peaks = load_peaks()           # position, H3K27ac, H3K4me3, ATAC
2 peaks.head(5)
```

	position	H3K27ac	H3K4me3	ATAC
0	0.00000	0.069866	0.048841	0.000581
1	20.02002	0.053866	0.048752	0.019810
2	40.04004	0.054921	0.019941	0.081723
3	60.06006	0.014632	0.008636	0.053283
4	80.08008	0.093176	0.043320	0.021620

H3K27ac marks active enhancers, H3K4me3 marks active promoters, ATAC marks open chromatin. Together they triangulate “where is something happening”.

```
1 genes = load_peak_genes()      # name, start, end, strand
2 genes.head(5)
```

	name	start	end	strand
0	GeneA	1500	4500	+
1	GeneB	7000	10500	-
2	GeneC	13500	17000	+

Peaks · Step 3 — Three Stacked Signal Tracks

PEAKS · BUILD

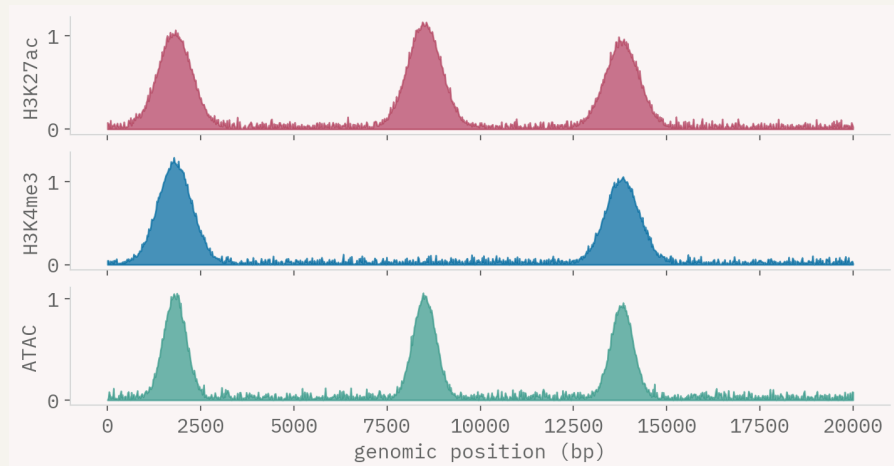
3 · One mark per panel, shared x-axis

Three filled-area panels stacked vertically with

`sharex=True`. The shared x-axis is the contract —

column-aligned positions across all panels let the eye spot a triple co-occurrence (active promoter + active enhancer + open chromatin) at a single x-coordinate.

```
1 peaks = load_peaks()                # position + 3 sig
2 x = peaks["position"]                # shared genomic i
3
4 # CREATE three panels stacked, all sharing the x-axis
5 fig, axes = plt.subplots(
6     3, 1, figsize=(8.8, 4.2), sharex=True,
7     gridspec_kw=dict(hspace=0.15))
8 ax_ac, ax_k4, ax_at = axes           # name each panel
9
10 # top panel: H3K27ac — active enhancer mark, RED
11 ax_ac.fill_between(x, peaks["H3K27ac"], where=peaks["H3K27ac"] > 0,
12                   color=RED, alpha=0.7) # CALL: filled area
13 ax_ac.set_ylabel("H3K27ac"); ax_ac.spines[["top", "right"]].set_visible(Fa
14
15 # middle panel: H3K4me3 — active promoter mark, BLUE
16 ax_k4.fill_between(x, peaks["H3K4me3"], where=peaks["H3K4me3"] > 0,
17                  color=BLUE, alpha=0.7)
18 ax_k4.set_ylabel("H3K4me3"); ax_k4.spines[["top", "right"]].set_visible(Fa
19
20 # bottom panel: ATAC — open chromatin, GREEN
21 ax_at.fill_between(x, peaks["ATAC"], where=peaks["ATAC"] > 0,
22                  color=GREEN, alpha=0.7)
```



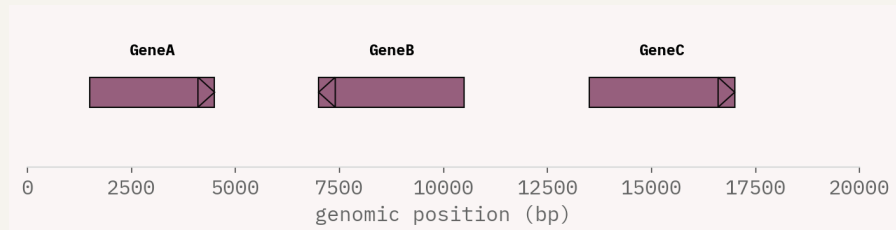
Peaks · Step 4 — Gene Model on Its Own

PEAKS · BUILD

4 · Rectangles for gene bodies, arrowheads for strand

Before we glue it to the signal tracks, get the **gene model** right on its own axis. Each gene is a **Rectangle** spanning **start → end**. Strand is encoded with a small **arrowhead** — a triangular **Polygon** pointing right for **+** strand, left for **-**.

```
1 from matplotlib.patches import Rectangle, Polygon
2
3 genes = load_peak_genes()           # name, start, end, strand
4 peaks = load_peaks()                # only used for x
5 xmin, xmax = peaks["position"].min(), peaks["position"].max()
6
7 fig, ax = plt.subplots(figsize=(8.8, 1.6))    # CREATE thin track
8
9 # helper to draw one gene as body + arrowhead + name
10 def draw_gene(ax, row):
11     y, h = 0, 0.4                    # vertical centre
12     body = Rectangle((row["start"], y - h / 2),    # rectangle anchor
13                     row["end"] - row["start"], h,    # width = gene length
14                     facecolor=PURPLE, edgecolor=INK, lw=0.8)
15     ax.add_patch(body)                # CALL: gene body
16     # strand-aware arrowhead at the leading edge of the gene
17     if row["strand"] == "+":
18         tip_x, base_x = row["end"], row["end"] - 400 # points right
19     else:
20         tip_x, base_x = row["start"], row["start"] + 400 # points left
21     head = Polygon([(base_x, y - h / 2), (base_x, y + h / 2), (tip_x, y)],
22                   facecolor=PURPLE, edgecolor=INK, lw=0.8)
```



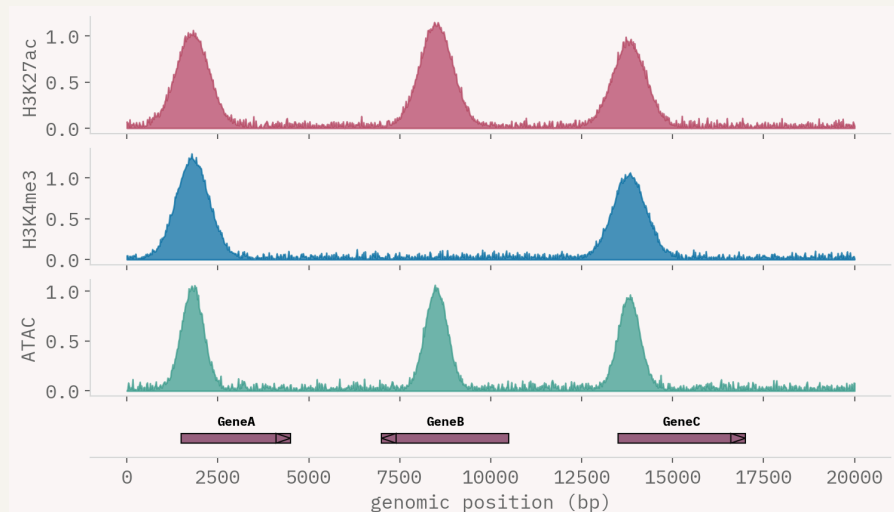
Peaks • Step 5 — Four-Panel Paper Figure

PEAKS • BUILD

5 • Three signal panels + one gene model, one shared x-axis

Glue everything together: **four vertical panels** sharing one x-axis — H3K27ac, H3K4me3, ATAC, then a thin gene-model panel on the bottom. The gene track gets a `height_ratios` shrink and minimal styling (no y-ticks, no frame) so it reads as an annotation, not a fourth data series.

```
1 from matplotlib.patches import Rectangle, Polygon
2
3 peaks = load_peaks(); genes = load_peak_genes()
4 x = peaks["position"]
5
6 # CREATE four panels — three signal, one gene model. gene panel is shorter
7 fig, axes = plt.subplots(
8     4, 1, figsize=(9.0, 5.0), sharex=True,
9     gridspec_kw=dict(height_ratios=[1, 1, 1, 0.4], hspace=0.15))
10 ax_ac, ax_k4, ax_at, ax_g = axes
11
12 # three signal panels (same as step 3, each gets a colour + label)
13 for ax, col, color in [(ax_ac, "H3K27ac", RED),
14                        (ax_k4, "H3K4me3", BLUE),
15                        (ax_at, "ATAC", GREEN)]:
16     ax.fill_between(x, peaks[col], where=peaks[col] > 0, color=color, alpha=0.5)
17     ax.set_ylabel(col); ax.spines[["top", "right"]].set_visible(False)
18
19 # gene-model panel — minimal frame so it reads as annotation
20 def draw_gene(ax, row):
21     y, h = 0, 0.5
22     ax.add_patch(Rectangle((row["start"], y - h / 2),
```



Peaks • Production Tool

PEAKS • ECOSYSTEM

pyGenomeTracks for paper-grade multi-track plots

For real screenshots — dozens of tracks, BAM files, BigWig, BED, GTF, Hi-C — use [pyGenomeTracks](#). You write a tiny `.ini` config and call one CLI command. It handles axes, scaling, gene model rendering, and Hi-C heatmaps that matplotlib can't touch.

```
1 [H3K27ac]
2 file = H3K27ac.bw
3 title = H3K27ac
4 color = #B4405F
5 height = 1.5
6
7 [genes]
8 file = annotation.gtf
9 title = genes
10 style = UCSC
```

```
1 pyGenomeTracks --tracks tracks.ini --region chr2:1000-21000 -o figure.pdf
```

The matplotlib build you just saw is the **mental model**;
pyGenomeTracks is the production tool.

Genome Stats • Takeaway

GENOME STATS • RECAP

Two stories, one principle

THE BOTTOM LINE

1. **Coverage tells you what you can trust.** A dumbbell sorted by loss surfaces the worst samples in one glance — bar charts of raw vs callable bury the story. 2. **Peak tracks + gene models tell you what's happening where.** Stack one panel per signal, one panel for the gene model, all sharing the same x-axis. 3. **The shared x-axis is the contract.** Column-aligned positions across panels are what let the reader spot co-occurrence — without that, the panels are just three unrelated charts.

Circular Genome Plots

PYCIRCLIZE · CIRCOS-STYLE

Why Circular?

PYCIRCLIZE · THEORY

When the x-axis bends back on itself

Most genome views are linear: position on the x-axis, value on the y. That works for one chromosome at a time, and Manhattan plots cheat by laying 22 of them end-to-end. But the moment the *relationships between* chromosomes are the story — a translocation that moves a block from chr3 to chr14, a regulator on chr1 that drives expression on chr19, a Hi-C contact map — a line breaks.

Bend the axis into a circle and three things change:

- Every chromosome becomes the **same visual length** to the eye, regardless of bp.
- You can draw a **chord** between any two points without crossing the figure.
- You can stack many **concentric tracks** (expression, methylation, density) without a vertical spaghetti of axes.

The trade-off: less detail *within* each chromosome (it's a curved sliver), more visibility *across* them. Reach for circular when the answer is in the connections, not the local peaks.

Anatomy of a Circos Figure

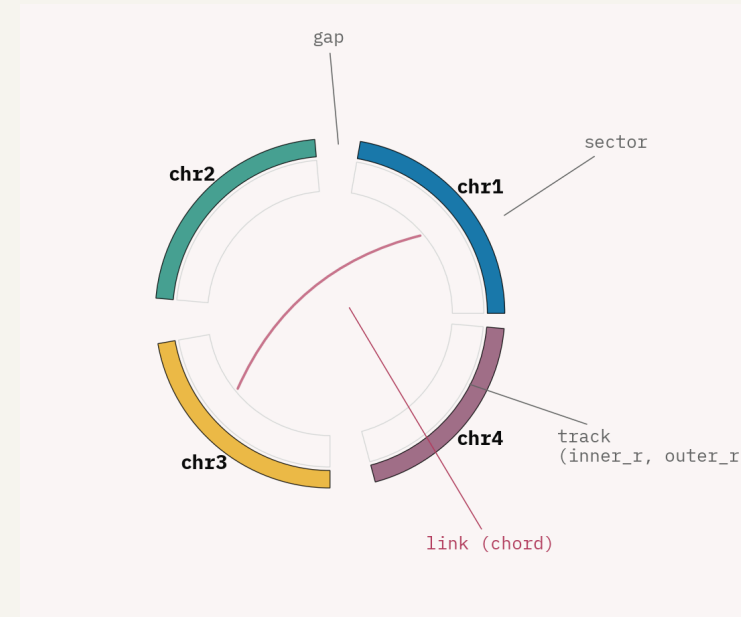
PYCIRCLIZE · THEORY

Sectors, tracks, links — the three primitives

A circular plot has three building blocks:

- **Sectors** — wedges of the ring, one per chromosome (or contig, or sample). The gap between them is a visual breath.
- **Tracks** — concentric bands at radii `(inner_r, outer_r)`. Each track holds one data layer: scatter, line, bar, heatmap.
- **Links** — chords drawn through the centre, joining two genomic intervals on possibly different sectors.

Build a Circos by declaring the sectors first (`{name: length}`), then adding tracks to each sector and links between them.



Circos • The Data

PYCIRCLIZE • DATA PEEK

Same GWAS table, this time on a ring

`load_gwas("height")` is the table we already met for the Manhattan: one row per SNP, four columns. To draw it circular, we need one extra dictionary — the **length of each chromosome** — which is just `max(pos)` per `chrom`. That dictionary defines the sectors.

```
1 gwas = load_gwas("height")           # chrom, pos, pval, snp
2 gwas.head(5)
```

```
1 # per-chromosome length = max position on that chromosome
2 chrom_len = gwas.groupby("chrom")["pos"].max().to_dict()
3 {k: chrom_len[k] for k in list(chrom_len)[:4]}    # peek at first four
```

	chrom	pos	pval	snp
0	1	562879	0.197842	rs1217797
1	1	802716	0.454291	rs3152883
2	1	933561	0.750287	rs1966161
3	1	1294717	0.707315	rs2836508
4	1	1316327	0.553459	rs6514392

{1: 249795092, 2: 249744166, 3: 249840488, 4: 249737521}

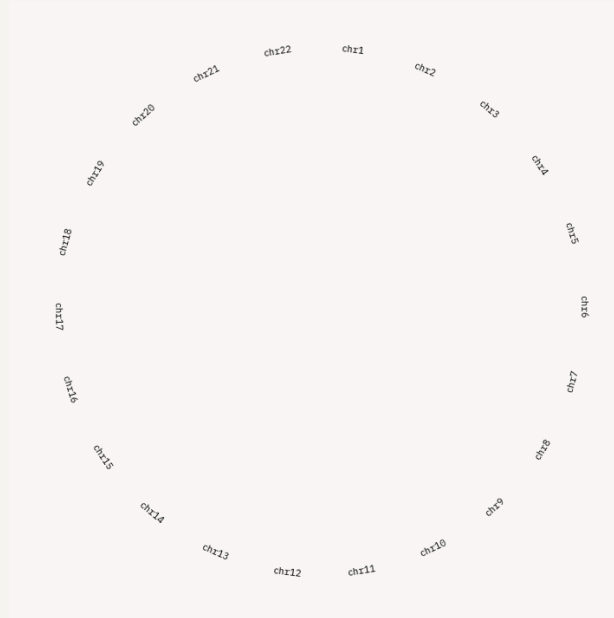
Circos • Step 1 — Just the Sectors

PYCIRCLIZE • BUILD

1 • The empty ring

Before any data, build the layout. The `sectors` dict maps chromosome name to length in bp; `pyCirclize` converts that into proportionally-sized wedges. We give each sector a label and draw — at this point the figure is a labelled empty ring, the canvas every other layer will rest on.

```
1 from pycirclize import Circos          # the main class
2
3 gwas = load_gwas("height")              # same table as Manhattan
4 chrom_len = gwas.groupby("chrom")["pos"].max().to_dict()
5
6 # sectors: {name: length_in_bp} — order = clockwise order on the ring
7 sectors = {f"chr{c}": chrom_len[c] for c in sorted(chrom_len)}
8
9 circos = Circos(sectors, space=2)        # CREATE: 2deg gap between
10
11 for sector in circos.sectors:
12     name = sector.name                  # pyCirclize: list of
13     sector.text(name, r=108, size=8, color=INK) # r = radius (axis is 0-10
14
15 fig = circos.plotfig(figsize=(6.5, 6.5)) # CALL: returns matplotlib
```



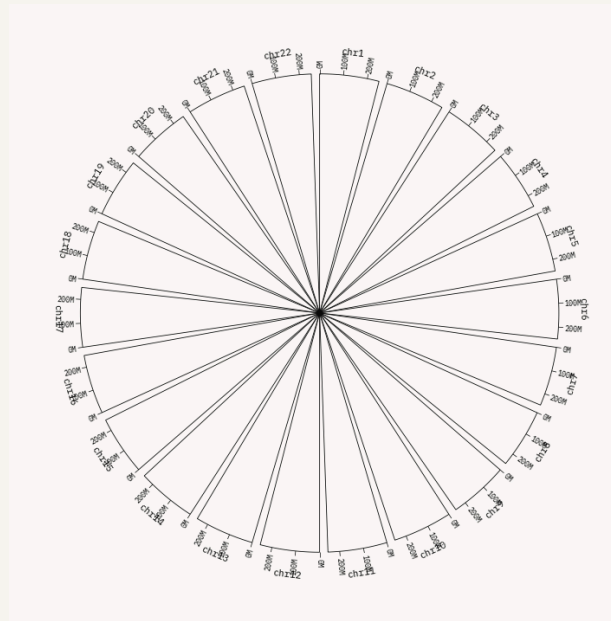
Circos • Step 2 — Axis & Ticks

PYCIRCLIZE • BUILD

2 • Give each sector a measuring stick

A ring with no marks is pretty but useless — we can't read positions. `sector.axis()` paints the outer boundary; `sector.xticks_by_interval(...)` drops tick marks at fixed bp intervals so the eye has anchors.

```
1 from pycirclize import Circos
2
3 gwas = load_gwas("height")
4 chrom_len = gwas.groupby("chrom")["pos"].max().to_dict()
5 sectors = {f"chr{c}": chrom_len[c] for c in sorted(chrom_len)}
6
7 circos = Circos(sectors, space=2) # CREATE
8
9 for sector in circos.sectors:
10     name = sector.name # pyCirclize: list
11     sector.text(name, r=108, size=8, color=INK)
12     # outer axis ring with line styling
13     sector.axis(fc=CREAM, ec=INK, lw=0.6)
14     # ticks live on a Track, not on the Sector itself — make a thin rim track
15     # the outer edge so the labels orbit the ring without overlapping data
16     rim = sector.add_track((96, 100))
17     rim.xticks_by_interval(100_000_000, #
18                             label_size=6, label_orientation="vertical", #
19                             label_formatter=lambda v: f"{v/1e6:.0f}M") #
20
21 fig = circos.plotfig(figsize=(6.5, 6.5)) # CALL
```



Circos • Step 3 — Manhattan-on-a-Ring

PYCIRCLIZE • BUILD

3 • Skyscrapers, pointing outward

Add a scatter track at radii (60, 90) — a band between 60% and 90% of the ring. For each sector, pull just that chromosome's SNPs out of the GWAS frame and scatter `pos` against `-log10(p)`. Significance now points outward: the higher the dot, the closer to the sky.

```
1 from pycirclize import Circos
2
3 gwas = load_gwas("height").copy()
4 gwas["nlog10p"] = -np.log10(gwas["pval"]) # significance axis
5 chrom_len = gwas.groupby("chrom")["pos"].max().to_dict()
6 sectors = {f"chr{c}": chrom_len[c] for c in sorted(chrom_len)}
7
8 circos = Circos(sectors, space=2) # CREATE
9
10 for sector in circos.sectors:
11     name = sector.name # pyCirclize: list
12     sector.text(name, r=108, size=8, color=INK)
13     sector.axis(fc=CREAM, ec=INK, lw=0.6)
14
15     # add a track at radii 60-90 (the band the SNP cloud lives in)
16     track = sector.add_track((60, 90))
17     # subset the dataframe to this chromosome only
18     chrom_num = int(name.replace("chr", ""))
19     df = gwas[gwas["chrom"] == chrom_num]
20     # CALL: scatter SNP position vs -log10(p) inside the track
21     track.scatter(df["pos"].to_numpy(), df["nlog10p"].to_numpy(),
22                  s=1.5, color=GREY, alpha=0.6)
```



Circos • Step 4 — Hits & Threshold Ring

PYCIRCLIZE • BUILD

4 • Colour what crossed the line

The same logic as the linear Manhattan: a dashed ring at $-\log_{10}(5e-8)$, and the SNPs above it re-drawn in red on top of the grey cloud. The threshold becomes a circle; significance becomes everything that pokes through it.

```
1 from pycirclize import Circos
2
3 gwas = load_gwas("height").copy()
4 gwas["nlog10p"] = -np.log10(gwas["pval"])
5 chrom_len = gwas.groupby("chrom")["pos"].max().to_dict()
6 sectors = {f"chr{c}": chrom_len[c] for c in sorted(chrom_len)}
7 gw_thresh = -np.log10(5e-8) # genome-wide signif.
8
9 circos = Circos(sectors, space=2) # CREATE
10
11 for sector in circos.sectors:
12     name = sector.name # pyCirclize: list
13     sector.text(name, r=108, size=8, color=INK)
14     sector.axis(fc=CREAM, ec=INK, lw=0.6)
15
16     track = sector.add_track((60, 90))
17     chrom_num = int(name.replace("chr", ""))
18     df = gwas[gwas["chrom"] == chrom_num]
19     # background cloud - grey
20     track.scatter(df["pos"].to_numpy(), df["nlog10p"].to_numpy(),
21                 s=1.5, color=GREY, alpha=0.6)
22     # hits - significant SNPs re-drawn red, larger
```



Circos • Step 5 — Chord Links

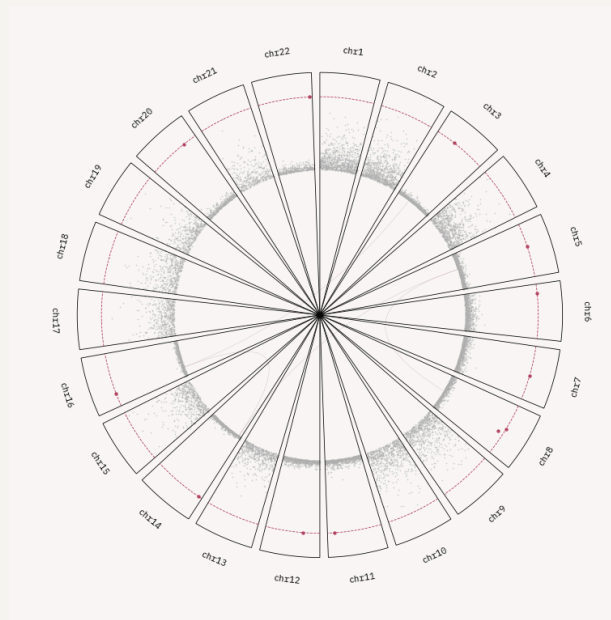
PYCIRCLIZE • BUILD

5 • Connect the hits across chromosomes

The headline feature of any Circos plot: **chords**.

`circos.link((sector_a, start, end), (sector_b, start, end))` draws a quadratic Bezier through the centre joining two intervals. We use it here to connect the top-5 most significant SNPs to each other — real publications use the same primitive for translocations, synteny blocks, or cis-/trans-regulatory pairs.

```
1 from pycirclize import Circos
2
3 gwas = load_gwas("height").copy()
4 gwas["nlog10p"] = -np.log10(gwas["pval"])
5 chrom_len = gwas.groupby("chrom")["pos"].max().to_dict()
6 sectors = {f"chr{c}": chrom_len[c] for c in sorted(chrom_len)}
7 gw_thresh = -np.log10(5e-8)
8
9 circos = Circos(sectors, space=2) # CREATE
10
11 for sector in circos.sectors:
12     name = sector.name # pyCirclize: list
13     sector.text(name, r=108, size=8, color=INK)
14     sector.axis(fc=CREAM, ec=INK, lw=0.6)
15     track = sector.add_track((60, 90))
16     chrom_num = int(name.replace("chr", ""))
17     df = gwas[gwas["chrom"] == chrom_num]
18     track.scatter(df["pos"].to_numpy(), df["nlog10p"].to_numpy(),
19                 s=1.5, color=GREY, alpha=0.6)
20     hits = df[df["pval"] < 5e-8]
21     if not hits.empty:
22         track.scatter(hits["pos"].to_numpy(), hits["nlog10p"].to_numpy(),
```



Circos • Takeaway

PYCIRCLIZE • RECAP

Three primitives, any biology

THE RECIPE

1. **Sectors** — `{name: length}` defines the ring; one wedge per chromosome. 2. **Tracks** — `sector.add_track((inner_r, outer_r))` is a concentric data band; stack as many as the eye can read. 3. **Links** — `circos.link(...)` is a chord between two intervals; the answer to “which moved with which”.

Circular plots trade per-chromosome zoom for cross-chromosome reach. Use them when the story is **relationships, not values** — translocations, synteny, Hi-C contacts, regulator-target pairs. Same logic for bacterial genomes (one sector per replicon), for metagenomes (one sector per MAG), even for protein domains (one sector per chain).

Synteny Plots

GENOME COMPARISON · RIBBONS

What Is a Synteny Plot?

SYNTENY • THEORY

Two genomes as tracks, ribbons as homology

A synteny plot stacks two (or more) chromosomes as horizontal tracks and connects matching regions with a **ribbon**. The geometry tells the story at a glance:

- **SYN** — block in the same order on both genomes. The ribbon is a flat parallelogram. Conserved synteny is the *uneventful* answer.
- **INV** — same region, opposite strand. The ribbon **twists** because we draw the bottom edge with its endpoints swapped. Inversions are common around centromeres and recombination cold spots.
- **TRANS** — block at a different position on the other genome. The ribbon crosses or skews; the region is *somewhere else* now.

Reach for synteny when you have two assemblies and the question is structural: what stayed put, what flipped, what moved.

Today vs Day-1's `plotsr`

SYNTENY · POSITIONING

Two complementary views of the same problem

On **Day 1** we walked through `plotsr`: the CLI pipeline `align → SyRI → plot`, the `genomes.txt`, the `*syri.out` files, the configurable multi-genome figure. It is the production tool.

Today we drop one level lower. We will draw a synteny figure for two genomes **in pure matplotlib**, from `Rectangle` patches and `Polygon` ribbons. No SyRI, no `plotsr` — just primitives and a small `DataFrame`.

After this section, `plotsr` will feel like sugar: it is doing the same thing for `N` genomes that you are doing here for `2`. Once you have built the ribbons yourself, you know which knob to turn when the published default does not fit your figure.

****Day 1 _15:**** `plotsr` — production CLI, multi-genome, SyRI input. ****Day 2 _07:**** `matplotlib` — two genomes, ribbons from polygons, full control.

Synteny • Build the Toy Data

SYNTENY • DATA

Six SYN blocks, one inversion, one translocation

We need a DataFrame with the homology calls. Each row is one block: where it lives on genome A, where it lives on genome B, and what type it is. The trick that encodes an inversion is dead simple — the B-coordinates are reversed (`chrB_start > chrB_end`), so when we later draw the polygon, the bottom edge twists.

```
1 # two "chromosomes", each 100 Mb in arbitrary units
2 LEN_A = 100.0
3 LEN_B = 100.0
4
5 # hand-built homology table: 6 SYN, 1 INV, 1 TRANS
6 blocks = pd.DataFrame([
7     # SYN: A-coords and B-coords ascending, near-identical positions
8     {"chrA_start": 5, "chrA_end": 18, "chrB_start": 6, "chrB_end": 19, "type": "SYN"},
9     {"chrA_start": 22, "chrA_end": 30, "chrB_start": 23, "chrB_end": 31, "type": "SYN"},
10    {"chrA_start": 34, "chrA_end": 45, "chrB_start": 35, "chrB_end": 46, "type": "SYN"},
11    {"chrA_start": 58, "chrA_end": 68, "chrB_start": 59, "chrB_end": 69, "type": "SYN"},
12    {"chrA_start": 72, "chrA_end": 80, "chrB_start": 73, "chrB_end": 81, "type": "SYN"},
13    {"chrA_start": 92, "chrA_end": 98, "chrB_start": 93, "chrB_end": 99, "type": "SYN"},
14    # INV: same span on A, but B-coords are reversed -> flipped strand
15    {"chrA_start": 48, "chrA_end": 56, "chrB_start": 56, "chrB_end": 48, "type": "INV"},
16    # TRANS: a block at A=82..90 lives at B=10..18 (somewhere else entirely)
17    {"chrA_start": 82, "chrA_end": 90, "chrB_start": 10, "chrB_end": 18, "type": "TRANS"},
18 ])
19 blocks.head(8)
```

	chrA_start	chrA_end	chrB_start	chrB_end	type
0	5	18	6	19	SYN
1	22	30	23	31	SYN
2	34	45	35	46	SYN
3	58	68	59	69	SYN
4	72	80	73	81	SYN
5	92	98	93	99	SYN
6	48	56	56	48	INV
7	82	90	10	18	TRANS

```
1 blocks["type"].value_counts() # quick sanity check on the mix
```

```
type
SYN      6
INV      1
TRANS    1
Name: count, dtype: int64
```

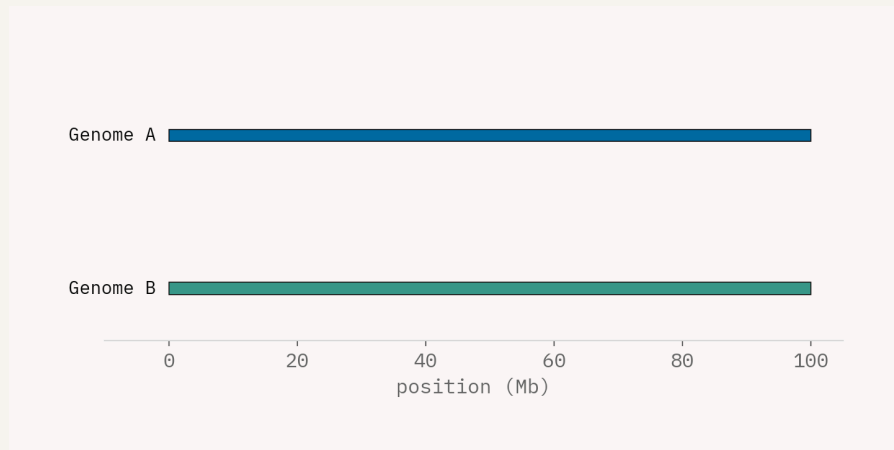
Synteny · Step 1 — The Two Chromosomes

SYNTENY · BUILD

1 · Two rectangles, one above the other

Before any ribbons, place the two chromosomes. They are just `Rectangle` patches: bottom-left corner at `(0, y)`, width = chromosome length, a thin height. Genome A goes on top, genome B on the bottom; the ribbons will eventually live in the gap between them.

```
1 from matplotlib.patches import Rectangle      # the chromosome bars
2
3 LEN_A, LEN_B = 100.0, 100.0                  # both 100 Mb units
4 Y_TOP, Y_BOT, BAR_H = 1.0, 0.0, 0.08        # y-positions and bar height
5
6 fig, ax = plt.subplots(figsize=(8, 3))        # CREATE
7
8 # genome A — top bar
9 ax.add_patch(Rectangle((0, Y_TOP), LEN_A, BAR_H,
10                        facecolor=BLUE, edgecolor=INK, lw=0.6)) # CALL
11 # genome B — bottom bar
12 ax.add_patch(Rectangle((0, Y_BOT), LEN_B, BAR_H,
13                        facecolor=GREEN, edgecolor=INK, lw=0.6)) # CALL
14
15 # label each bar on the left
16 ax.text(-2, Y_TOP + BAR_H / 2, "Genome A",
17         ha="right", va="center", fontsize=11, color=INK)
18 ax.text(-2, Y_BOT + BAR_H / 2, "Genome B",
19         ha="right", va="center", fontsize=11, color=INK)
20
21 # cosmetic frame
22 ax.set_xlim(-10, max(LEN_A, LEN_B) + 5); ax.set_ylim(0.2, 1.5)
```



Synteny · Step 2 — One Ribbon, By Hand

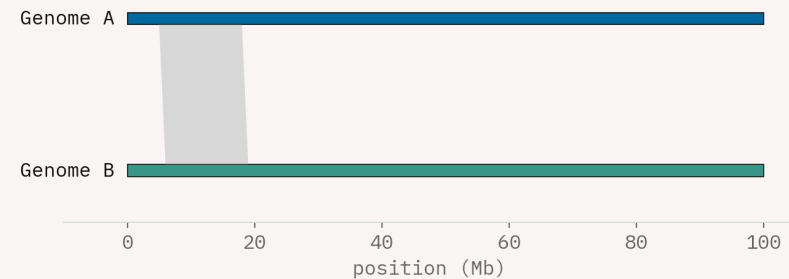
SYNTENY · BUILD

2 · A polygon with four vertices

A ribbon is a four-cornered polygon. The vertices go **clockwise**: top-left, top-right, bottom-right, bottom-left.

Top edge sits on genome A; bottom edge sits on genome B. Pick the first SYN block and draw exactly one ribbon to feel the geometry.

```
1 from matplotlib.patches import Rectangle, Polygon
2
3 LEN_A, LEN_B = 100.0, 100.0
4 Y_TOP, Y_BOT, BAR_H = 1.0, 0.0, 0.08
5
6 fig, ax = plt.subplots(figsize=(8, 3))          # CREATE
7
8 ax.add_patch(Rectangle((0, Y_TOP), LEN_A, BAR_H,
9                        facecolor=BLUE, edgecolor=INK, lw=0.6))
9
10 ax.add_patch(Rectangle((0, Y_BOT), LEN_B, BAR_H,
11                       facecolor=GREEN, edgecolor=INK, lw=0.6))
12
13 # pick the first block: A=5..18, B=6..19, SYN
14 b = {"chrA_start": 5, "chrA_end": 18, "chrB_start": 6, "chrB_end": 19}
15
16 # vertex order — clockwise starting at top-left:
17 # 1. (A_start, top edge of B-A gap = Y_TOP)
18 # 2. (A_end, top)
19 # 3. (B_end, bottom edge = Y_BOT + BAR_H)
20 # 4. (B_start, bottom)
21 verts = [(b["chrA_start"], Y_TOP),                # top-left
22          (b["chrA_end"], Y_TOP),                  # top-right
```



Synteny · Step 3 — All Blocks, Coloured by Type

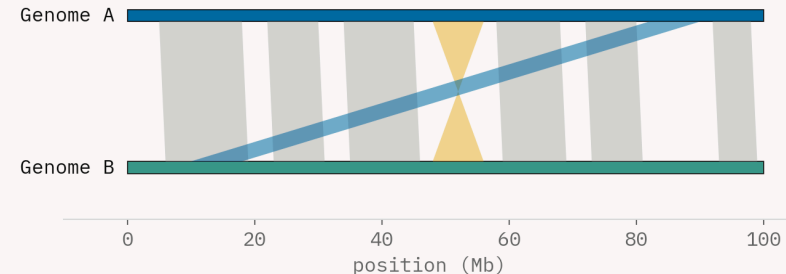
SYNTENY · BUILD

3 · The canonical figure

Now sweep through the whole DataFrame. The polygon shape is identical for every block — vertex order **top-left, top-right, bottom-right, bottom-left** — but the *meaning* changes with `type`:

- For **SYN** (grey), `chrB_start < chrB_end`, so the bottom edge goes left-to-right, same as the top — a flat parallelogram.
- For **INV** (amber), `chrB_start > chrB_end`, so the bottom edge reverses, creating a visual twist.
- For **TRANS** (blue), the B-coordinates live somewhere unrelated to the A-coordinates, so the ribbon skews across the figure.

```
1 from matplotlib.patches import Rectangle, Polygon
2
3 # colour map — one fill per block type
4 TYPE_COLOUR = {"SYN": GREY, "INV": AMBER, "TRANS": BLUE}
5
6 LEN_A, LEN_B = 100.0, 100.0
7 Y_TOP, Y_BOT, BAR_H = 1.0, 0.0, 0.08
8
9 fig, ax = plt.subplots(figsize=(8, 3))          # CREATE
10
11 ax.add_patch(Rectangle((0, Y_TOP), LEN_A, BAR_H,
12                        facecolor=BLUE, edgecolor=INK, lw=0.6))
13 ax.add_patch(Rectangle((0, Y_BOT), LEN_B, BAR_H,
14                        facecolor=GREEN, edgecolor=INK, lw=0.6))
15
16 # one polygon per row of the blocks DataFrame — vectorise via itertuples
17 for b in blocks.itertuples(index=False):
18     verts = [(b.chrA_start, Y_TOP),          # top-left
19             (b.chrA_end, Y_TOP),             # top-right
```



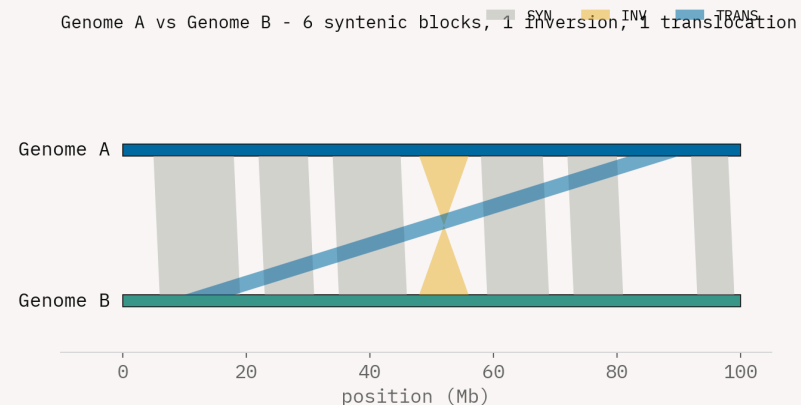
Synteny · Step 4 — Legend, Ticks, Title

SYNTENY · BUILD

4 · Polish — orient the reader

The figure is informative but spare. Add a tick every 20 Mb so the eye can read positions, a legend mapping colour to block type, and a title that names what we are looking at.

```
1 from matplotlib.patches import Rectangle, Polygon
2
3 TYPE_COLOUR = {"SYN": GREY, "INV": AMBER, "TRANS": BLUE}
4 LEN_A, LEN_B = 100.0, 100.0
5 Y_TOP, Y_BOT, BAR_H = 1.0, 0.0, 0.08
6
7 fig, ax = plt.subplots(figsize=(8, 3.4))          # CREATE
8
9 ax.add_patch(Rectangle((0, Y_TOP), LEN_A, BAR_H,
10                        facecolor=BLUE, edgecolor=INK, lw=0.6))
11 ax.add_patch(Rectangle((0, Y_BOT), LEN_B, BAR_H,
12                        facecolor=GREEN, edgecolor=INK, lw=0.6))
13
14 for b in blocks.itertuples(index=False):
15     verts = [(b.chrA_start, Y_TOP),
16             (b.chrA_end, Y_TOP),
17             (b.chrB_end, Y_BOT + BAR_H),
18             (b.chrB_start, Y_BOT + BAR_H)]
19     ax.add_patch(Polygon(verts, closed=True,
20                          facecolor=TYPE_COLOUR[b.type],
21                          edgecolor="none", alpha=0.55))
22
```



Synteny · Takeaway

SYNTENY · RECAP

The whole figure is one **PoLygon** call in a loop

THE RECIPE

1. **Two** **Rectangle** **bars** — one per genome, parallel and labelled.
2. **One** **PoLygon** **per block**, four vertices in clockwise order. Top edge on A, bottom edge on B.
3. **Vertex order encodes the rearrangement**. Bottom edge reversed = inversion; bottom edge far from top = translocation.
4. **Colour encodes the type**. Grey for conserved, amber for flipped, blue for moved.

Once you can draw this in matplotlib, packages like **plotsr** (Day 1) and **pyCirclize** feel like syntactic sugar over the same idea — they stack the bars, parse the homology table, and draw the polygons for you. Knowing the primitive means you can debug or customise any of them when the default does not say what your figure needs to say.